DeribitVerdictEngine

User Manual

Technical Reference

Generated May 13, 2026

BTC-PERPETUAL scalping analysis engine • Deribit REST API • VB.NET / .NET 8

Contents

1 DeribitVerdictEngine — User Manual

1.1 Introduction

DeribitVerdictEngine is a VB.NET / .NET 8 Windows Forms desktop application that polls the Deribit REST API on BTC-PERPETUAL, runs a multi-tier technical indicator pipeline on the live data, and emits a directional verdict with supporting diagnostics. It is an analysis and decision-support tool, not an execution system — it does not send orders.

This manual is a field-by-field reference for every variable and display block the engine writes to its RTF output pane. It assumes familiarity with BTC perpetual scalping, order flow, volume profile, and the trader profile in `docs/trader-profile.md`. For pipeline architecture and scoring steps, see `docs/architecture.md` and `docs/DeribitIndicatorProject.md`.

How to read the output (top to bottom):

1. **Verdict block** — the headline call, context qualifier, confidence tier, and raw/effective scores vs the regime-adjusted ceiling.
2. **ATR entry levels and Kelly sizing** — advisory reference frame for volatility context and directional conviction. See Kelly note below.
3. **Indicator sections** — each primitive's raw values and classification, ordered by tier (core → structural → microstructure → gates).
4. **Signal breakdown table** — the itemised scoring ledger. Reconciles every `[L]` / `[S]` hit and penalty against the TOTAL row, which matches the header's `SCORE`.

Important on Kelly sizing. The Kelly block is **advisory only** and uses an **ATR-basis** payoff ratio (`target_mult / stop_mult`, default `1.67`). Per the trader profile, real execution uses **structural** stops and targets (previous swing low/high), not ATR multiples — so the displayed Kelly fraction does not correspond to your actual R:R. Treat Kelly as a directional-bias sanity check, not a position-sizing prescription. The advisory label is rendered inline under the KELLY SIZING header to reinforce this.

Source of truth. When this manual and the code disagree, the code wins. Primary source files: `UI/MainForm_Render_Header.vb`, `UI/MainForm_Render_Sections.vb`, `Core/ScoringEngine_*.vb`, `Core/Indicators_*.vb`, `analysis/*.vb`, `tools/AutoTweaker/*.vb`, and `settings.json` v24 (auto-bumped by AutoTweaker after applied tweaks).

1.2 Table of Contents

1.3 1. Verdict Block

The header panel at the top of each analysis run. Always present.

```
============================================================
  VERDICT:    WEAK LONG
  CONTEXT:    MOMENTUM_FADING
  CONFIDENCE: LOW
  SCORE:      Long 8/15 (eff.7)  |  Short 2/15 (eff.1)  |  TRANSITIONAL penalty: -1
  TIME:       2026-04-22 00:39:42 UTC+8
============================================================
```

```
LAST TRANSACTED PRICE:  76038.0
HOLD / EXIT: EXIT -- momentum break (ROC crossed below 0)
```

1.3.1 VERDICT

What: Final directional call from the scoring pipeline.

Calculation: In `ScoringEngine_Calculate_Verdict.vb` Step 5. After all scoring passes, funding modifiers, and veto gates produce `effectiveLS` and `effectiveSS`, these are compared against percentage thresholds of the regime-adjusted `MaxScore`:

- `tStrong = ceil(MaxScore × verdict_strong_pct)` — default 0.70
- `tMed = ceil(MaxScore × verdict_med_pct)` — default 0.53
- `tWeak = ceil(MaxScore × verdict_weak_pct)` — default 0.35

Long side is evaluated first; if long passes no tier, short is evaluated.

Possible values:

Verdict	Trigger
`STRONG LONG`	`effectiveLS` $\geq$ `tStrong`
`LONG`	`tMed` $\leq$ `effectiveLS` < `tStrong`
`WEAK LONG`	`tWeak` $\leq$ `effectiveLS` < `tMed`
`STRONG SHORT`	`effectiveSS` $\geq$ `tStrong` AND short wins
`SHORT`	`tMed` $\leq$ `effectiveSS` < `tStrong` AND short wins
`WEAK SHORT`	`tWeak` $\leq$ `effectiveSS` < `tMed` AND short wins
`NO TRADE`	Neither side clears `tWeak`
`NO TRADE [WEAK LONG]` / `NO TRADE [WEAK SHORT]`	Regime veto (Step 4) or MTF Gate block (Step 4b) triggered, but the underlying score still has a weak lean. The bracketed tag is the suppressed direction.

Interpretation: Trader-profile rule is act on `MEDIUM` / `HIGH` tiers only. `WEAK` readings are informational — they mean a partial setup exists but the cross-confirmation density is insufficient. `NO TRADE [WEAK X]` means "the scoring said X but a hard gate killed it" — the lean is real but you're fighting either regime or the 15m timeframe. Stand down.

1.3.2 CONTEXT

What: Quality qualifier on the verdict. **Conditionally displayed** — only shown when `VerdictContext != "CONFIRMED"`. If absent, the read is clean.

Calculation: In `ScoringEngine_Calculate_Scoring.vb::CalcVerdictContext`. Classifies the dominant side using three audits:

1. **Fading count.** Scores hits from: `MicroCVDSignal = BULL_DECEL/BEAR_DECEL`, `TTMSignal = BULL_FADING/BEAR_FADIN` RSI beyond divergence penalty threshold (>65 or <35), and MicroCVDLate collapsing to ≤50% of MicroCVDEarly on the dominant side. ≥ 2 hits → `MOMENTUM_FADING`.
2. **Structural / flow split.** Counts structural breakdown hits (VWAP, BBW/TTM, EMA ribbon, DMI, ADX, Donchian, 5m EMA200) vs flow hits (OFI, CVD, TFI, MicroCVD, OI Delta, ROC, Volume). Structural ≥ `ContextTagStructuralMin` (3) AND flow ≤ `ContextTagFlowMax` (1) → `FLOW_UNCONFIRMED`.
3. **Thin signal set.** Both structural < 2 AND flow < 2 → `STRUCTURALLY_WEAK`.
4. Otherwise → `CONFIRMED` (line suppressed).

Possible values:

Context	Meaning
`MOMENTUM_FADING`	Dominant side's momentum primitives are rolling over. Price can still print the verdict direction but the driving force is weakening.
`FLOW_UNCONFIRMED`	Clean structural setup with no order-flow backing. Often the precursor to a stall or fakeout.
`STRUCTURALLY_WEAK`	Neither structure nor flow has enough signal density. The verdict exists but rests on very little.
`CONFIRMED` *(hidden)*	Balanced hits on both axes; no fading flags.

Interpretation: This is a *veto layer for human judgement*, not for the engine — it does not alter the score. A `WEAK LONG` with `MOMENTUM_FADING` is borderline; the same verdict with line absent is cleaner. Use it to choose between "pass" and "enter cautious".

1.3.3 CONFIDENCE

What: Tier label corresponding to which threshold the dominant effective score cleared.

Calculation:

Confidence	Set by
HIGH	effectiveScore ≥ tStrong
MEDIUM	effectiveScore ≥ tMed and < tStrong
LOW	effectiveScore ≥ tWeak and < tMed
N/A	Verdict is NO TRADE (no tier cleared, or veto/MTF block fired)

Interpretation: Maps 1:1 to the verdict tier — exists for at-a-glance readability rather than extra information. N/A on a NO TRADE [WEAK X] means the veto path was taken, not that score was zero.

1.3.4 SCORE

What: Raw and effective score totals for both sides, against the regime-adjusted ceiling.

Calculation:

- LongScore / ShortScore — the raw ls / ss values after Step 2, all Pass 2 upgrades, Pass 2b OI×CVD, Pass 2c regime alignment, and Steps 3/3b funding modifiers. These are the numbers shown in the breakdown table TOTAL row.
- EffectiveLongScore / EffectiveShortScore — raw scores after Step 4 regime veto adjustments. In TRANSITIONAL, an ADX-graded penalty is applied (floored by TierFloor(rawScore)). In TRENDING_UP/TRENDING_DOWN, these equal the raw scores (regime veto is an all-or-nothing NO TRADE, not a graded reduction).
- MaxScore — regime ceiling: TRENDING 20, RANGE_BOUND 19, TRANSITIONAL 15 (under default regime_weights.enabled = true). Subtract the alignment bonus (1) if the gate is disabled to get the pre-v13 values 19/18/15.

Display logic:

- **Normal form:** Long N/M | Short N/M — used when RegimePenalty = 0.
- **Penalty form:** Long N/M (eff.E) | Short N/M (eff.E) | TRANSITIONAL penalty: -P — used when RegimePenalty > 0. The penalty magnitude P is sourced from regime_gates.transitional_penalty_low (ADX 20–22.5) or transitional_penalty_mid (ADX 22.5–25).

Interpretation: The TierFloor guard matters. When raw score is high, the penalty is fully applied; when raw is already low, the floor prevents over-penalising — so a WEAK verdict at the floor is not as weak as it looks. Eye the delta between raw and effective: a large delta in TRANSITIONAL means the ADX is genuinely borderline and should inform position sizing independently of the verdict tier.

1.3.5 TIME

What: Local timestamp (UTC+8, hardcoded for the trader's timezone) when this run executed.

Calculation: `DateTime.UtcNow.AddHours(8)` formatted `yyyy-MM-dd HH:mm:ss UTC+8`. Not the time the data was captured by Deribit — the time your local process rendered.

Interpretation: Use to verify auto-run freshness and to align verdicts with session boundaries (Asia 00:00–07:59 UTC = 08:00–15:59 UTC+8; London 08:00–12:59 UTC = 16:00–20:59 UTC+8; NY 13:00–23:59 UTC = 21:00 UTC+8 – 07:59 next day UTC+8).

1.3.6 LAST TRANSACTED PRICE

What: Price of the most recent trade fetched from `GetRecentTradesAsync(100)`, above the ATR block.

Calculation: `recentTrades[0].Price`. Displays `N/A` if the recent-trades fetch returned empty.

Interpretation: This is **not** the entry price used in ATR levels — that uses the close of the last 1m candle (`r.CurrentPrice`). Compare the two to spot slippage between the closed candle and the live tape. A large divergence in the second or two since the candle closed signals an in-progress impulse.

1.3.7 HOLD / EXIT

What: Guidance line for an open position. **Conditionally displayed** — only shown when `HoldStatus != "N/A -- no open position"`. Requires the internal `posState` to be `InLong` or `InShort` (set via UI controls or programmatic state).

Calculation: `CalcHoldStatus` in `ScoringEngine_Helpers.vb`. Layered priority:

1. **Microstructure fast exit:** ≥ 2 adverse signals (MicroCVD DECEL + OFI opposing + TFI opposing + CVD opposing, side-dependent) → `EXIT -- microstructure deterioration (<list>)`.
2. **Structural breaks:** ROC crossing zero against the position → `EXIT -- momentum break`. OBV divergence against position → `EXIT -- OBV <X> divergence`. RSI divergence against position → `EVALUATE -- RSI <X> divergence`.
3. **Single adverse microstructure:** `EVALUATE -- <signal> signal, confirm with price action`.
4. **ROC/RSI structural assessment:** ROC beyond take-profit band → `TAKE PROFIT -- extreme momentum, tighten stops`. RSI above/below hold threshold → `HOLD -- momentum intact`. RSI in evaluate band → `EVALUATE -- momentum weakening, consider scaling out`. RSI past evaluate threshold → `EXIT -- retracement too deep (RSI < 40)` / `(RSI > 60)`.

All RSI/ROC thresholds sourced from `cfg.Scoring.HoldRoc*` and `HoldRsi*`.

Interpretation: Read top-down — the first matching layer wins. A fast-exit microstructure signal

means you're already late; a TAKE PROFIT means you're running an outlier extension and should tighten rather than chase. EVALUATE lines are cognitive prompts, not mandates — price action on the 1m is the arbiter.

1.4 2. ATR Entry Levels

```
ATR ENTRY LEVELS  (ATR 57.60 x 0.79 scale | 1.2x stop / 2.0x target)
 Long:  Stop  75983.1 | Entry  76038.0 | Target  76129.4  R:R 1:1.7 (risk 54.9 / rwd 91.4)
 Short: Stop  76092.9 | Entry  76038.0 | Target  75946.6  R:R 1:1.7 (risk 54.9 / rwd 91.4)
```

1.4.1 Section Header

Format: `ATR ENTRY LEVELS  (ATR <atr> x <scale> scale | <stopMult>x stop / <targetMult>x target)`

- `atr` — `r.ATR`, raw 7-period ATR on 1m candles.
- `scale` — `norms.ATRScaleFactor`, clamped ratio of current ATR to rolling reference (see Dynamic Norms below).
- `stopMult` — `cfg.Scoring.AtrStopMultiplier`, default 1.2.
- `targetMult` — `cfg.Scoring.AtrTargetMultiplier`, default 2.0.

All four read live from config; the label display is dynamic, not hardcoded. R:R implied is `targetMult / stopMult` (default 1:1.67 → rounded `1:1.7`).

1.4.2 Stop / Entry / Target (Long)

What: Long-direction trade frame.

Calculation:

- `atrStop = r.ATR × norms.ATRScaleFactor × stopMult`
- `atrTarget = r.ATR × norms.ATRScaleFactor × targetMult`
- `longStop = r.CurrentPrice - atrStop`
- `longTarget = r.CurrentPrice + atrTarget`
- Entry = `r.CurrentPrice` (close of the last 1m candle, not the live tape).

1.4.3 Stop / Entry / Target (Short)

Mirrored: `shortStop = r.CurrentPrice + atrStop`, `shortTarget = r.CurrentPrice - atrTarget`.

1.4.4 R:R / risk / rwd

- `R:R` — literal `targetMult / stopMult` string. Static for a given config; does not reflect realised outcomes.
- `risk` — `atrStop` (distance in price points to stop).
- `rwd` — `atrTarget` (distance in price points to target).

1.4.5 VPFR HVN-capped target (conditional)

Triggered when: A VPFR POC sits between entry and the raw ATR target AND the VPFR signal is classifying that POC as a relevant wall for the direction.

- Long cap fires when `v.AdjustedLongTarget > 0`, which is set in `ScoringEngine_Calculate_Verdict.vb` Step 5b when `VPFRSignal ∈ {NEAR_HVN_RESIST, IN_LVN_BEAR}` AND `VPFRPoc > CurrentPrice` AND `VPFRPoc < rawLongTarget`.
- Short cap fires on the mirror: `VPFRSignal ∈ {NEAR_HVN_SUPPORT, IN_LVN_BULL}` AND `VPFRPoc < CurrentPrice` AND `VPFRPoc > rawShortTarget`.

Display: Raw target printed dimmed, followed by `--> <adjusted>  [<reason>]` in amber bold. Reason field is `swing` (highest priority — see below) / `hvn` / `poc` / `none`. Logged as CSV column 84 `TargetCapReason`.

Step 5b 3-tier cap (v17 swing-pivot spec): the cap is now closest-wins across three tiers:

1. **Tier 1 — swing target.** `SwingTargetLong` if it sits between entry and the raw long target (or above the raw target by a smaller margin). Mirrored for short.
2. **Tier 2 — nearest HVN wall.** `VPFRNearestHvnAbove` for long, `VPFRNearestHvnBelow` for short.
3. **Tier 3 — POC fallback.** `VPFRPOC` when neither tier above qualifies AND VPFR signal indicates resistance / support against the verdict direction.

Closest cap to entry wins. The selected reason is recorded in `v.TargetCapReason` and surfaced in the display.

Interpretation: The cap tells you the raw 2×ATR target is probably unreachable because there's structure (a swing high, an HVN wall, or a POC) sitting between entry and target. Use the capped value as the realistic scale-out level, or skip the trade if the capped R:R no longer makes sense. This is an engine convenience, not a scoring input — it does not change the verdict.

Important divergence from trader-profile. These ATR levels are **display only** for sanity reference. Per `trader-profile.md` §4–5, live execution uses **structural** stops (previous swing low/high) and **structural** targets (previous swing high/low). The ATR frame is a volatility-context reference and, via the Kelly block, an advisory sizing basis. Do not place the ATR stop or target as your actual working orders.

1.5 3. Kelly Sizing

```
KELLY SIZING  [CAPPED]
  Advisory (ATR-basis) --- R:R uses ATR multiples, not structural targets.
  Treat as directional bias indicator only.
  p(win):   45.0%
  f* / Half-Kelly:  12.00%  /  6.00%
  Applied fraction: 5.00%
  Risk $:    $50.00
  Contracts: < 1 contract  (stop too wide for min size)
```

Rendering gate: Block only appears when `v.KellyPWin > 0`. `CalcKellySizing` early-exits (leaving all Kelly fields at 0) when verdict is empty/`NEUTRAL`/`WAIT`, or `stopDistanceUsd <= 0`, or `AtrStopMultiplier <= 0`. Under normal operation this means the block renders for every real verdict, including `NO TRADE`.

1.5.1 Section Header

Format:

- Normal: `KELLY SIZING` (optionally `[CAPPED]`)
- NO TRADE: `KELLY SIZING  [BIAS ONLY --- NO TRADE]` (optionally `[CAPPED]`)

`[CAPPED]` tag appears when `KellyCapped = true`, which is true when half-Kelly exceeded `MaxRiskFraction` (default 5%) and the applied fraction was clamped down.

`[BIAS ONLY --- NO TRADE]` tag appears when `v.Verdict.StartsWith("NO TRADE")`. In this mode Kelly is computed but labelled as direction-only — do not trade.

Historic note: Earlier versions also showed `[EST]` or `[CAL]` mode tags. CAL mode was removed (see v14 changelog); `KellyPMode` is still assigned `"EST"` internally but no longer rendered. Tag will return once a backtesting module supplies empirical win rates.

1.5.2 Advisory label (always present)

Two dim-grey lines immediately under the header:

```
Advisory (ATR-basis) --- R:R uses ATR multiples, not structural targets.
Treat as directional bias indicator only.
```

Fixed literal text. Rendered unconditionally whenever the Kelly block renders.

Interpretation: A deliberate reminder that the Kelly fraction is computed off the ATR R:R ratio (default 1.67), not the structural R:R the trader actually uses. Read the block as "the engine's directional conviction, translated into a sizing hint" rather than a prescription.

1.5.3 p(win)

What: Win probability used in the Kelly formula.

Calculation: `ScoringEngine_Kelly.vb`. EST-mode mapping of `Confidence` onto a probability band:

Confidence	p
`HIGH`	`EstProbFloor + EstProbScale` = 0.45 + 0.20 = **0.65**
`MEDIUM`	`EstProbFloor + EstProbScale / 2` = 0.45 + 0.10 = **0.55**
`LOW` / other	`EstProbFloor` = **0.45**

Floor and scale are `cfg.Kelly.EstProbFloor` (0.45) and `cfg.Kelly.EstProbScale` (0.20).

Interpretation: Pre-calibration tiering. These numbers are engineered priors, not measured outcomes. `HIGH` confidence asserts a 65% win prior — treat with scepticism until backtesting can replace these with observed rates.

1.5.4 f* / Half-Kelly

What: Raw Kelly fraction and its half-Kelly damped variant, as percentages.

Calculation:

- `b = cfg.Scoring.AtrTargetMultiplier / cfg.Scoring.AtrStopMultiplier` — default 2.0 / 1.2 = 1.667.
- `q = 1 − p`.
- `f* = (b × p − q) / b`. Negative means no edge under these priors.
- `fHalf = f* / 2` if `cfg.Kelly.UseHalfKelly` (default true), else `f*`.

If `f*` $\leq$ `0`, the entire block exits silently (see gate above).

Interpretation: Half-Kelly is a standard industry damping to trade the drawdown-vs-growth trade-off. `f*` above 20% with `p ≈ 0.65` is already aggressive; real-world edge is usually closer to `p ≈ 0.5`, which gives the common Kelly $\leq$ 0 "stand down" result.

1.5.5 Applied fraction

What: The fraction actually used for the Risk $ computation, after the hard cap.

Calculation: `fApplied = min(fHalf, cfg.Kelly.MaxRiskFraction)`. Default cap 0.05 (5%). `KellyCapped = fHalf > MaxRiskFraction`.

Interpretation: Any time you see `[CAPPED]`, the engine thinks you have more edge than the 5% safety cap permits. Don't try to override it — cap exists because pre-calibration p-values are unreliable upward.

1.5.6 Risk $

What: Dollar risk per trade at the applied fraction.

Calculation: `cfg.Kelly.AccountSizeUsd` × `fApplied`. Default account 1000 × 5% cap = $50 maximum.

Interpretation: `AccountSizeUsd` is a placeholder pending trader input. Scale mentally: the displayed Risk $ × (your real account / 1000) is the rough translation. Or just set the field correctly in `settings.json` and the math propagates.

1.5.7 Contracts / Lean

What: Recommended contract count, derived from dollar risk and ATR stop distance.

Calculation: `KellyContracts = floor(KellyRiskUsd / (ContractFaceUsd × stopDistanceUsd))`. `ContractFaceUsd` default 10.

Display variants:

Condition	Label	Value
Normal, `Contracts ≥ 1`	`Contracts:`	`N contracts`
Normal, `Contracts < 1`	`Contracts:`	`< 1 contract (stop too wide for min size)`
NO TRADE, `Contracts ≥ 1`	`Lean:`	`N contracts (not a trade signal)`
NO TRADE, `Contracts < 1`	`Lean:`	`< 1 contract (bias only; not a trade signal)`

Interpretation: `< 1 contract` on a normal verdict means the ATR stop distance is wide enough that even your full risk budget doesn't buy a single minimum contract — the trade is impractical at this volatility unless you widen the risk limit. In `BIAS ONLY` mode, any contract count is directional colour only; don't work the order.

Known formula limitation. Contract sizing uses `ContractFaceUsd × stopDistance` for risk-per-contract. For a true Deribit BTC-PERPETUAL (inverse), the correct risk formula includes the entry price in the denominator. This is a known simplification matching the approved spec and will drift from real contract PnL at extreme stops. Treat the contract count as ±1–2 rough.

1.6 4. Dynamic Norms

```
DYNAMIC NORMS  [LIVE]
  Vol threshold : H:3.96x  M:2.44x  (mean=6.2898 BTC  s=7.6716)
  VWAP dev thr  : +/-0.30% (legacy ref)
  ATR scale     : 0.79x  (ATR=57.60  ref=72.58)
```

The adaptive threshold layer (`DynamicNorms.vb`). Computed per run, applied to scoring thresholds downstream (Volume classifier, ATR-scaled targets/stops).

1.6.1 Section Header Tag

Format: `DYNAMIC NORMS  [LIVE]` or `DYNAMIC NORMS  [STATIC FALLBACK]`.

- **`[LIVE]`** — `DynamicNorms.Compute` produced results from the current 1m candle history ($\geq$ 30 candles, $\geq$ 10 volume samples).
- **`[STATIC FALLBACK]`** — cold start or insufficient history; values come from static constants in `cfg.Indicators.{Volume,VWAPDynamic,ATR}`.

Interpretation: `STATIC FALLBACK` after a long uptime means the Deribit API call failed or returned stale data — treat the verdict with suspicion until the next run flips back to `LIVE`.

1.6.2 Vol threshold

What: Dynamic upper and mid volume ratio thresholds used by the Volume signal in Step 2, and the `mean` / stdev `s` statistics underlying them.

Calculation: In `DynamicNorms.Compute`:

- `volMean` — rolling average of the last `min(100, candles - 1)` 1m candle volumes (BTC).
- `volSD` — population standard deviation over the same window.
- **Collapsed-variance fallback:** If `volSD < volMean × 0.05`, static values are used: `H = cfg.Indicators.Volume.StaticHigh` (3.0), `M = StaticMid` (2.0). This prevents division-by-noise when volume has flat-lined.
- **Normal case:**
  - `highRaw = (volMean + 2 × volSD) / volMean`
  - `midRaw  = (volMean + 1 × volSD) / volMean`
  - Each clamped to `[DynamicHighClampMin, DynamicHighClampMax] = [2.0, 6.0]` (v14) and `[DynamicMidClampMin, DynamicMidClampMax] = [1.5, 4.0]` (v14).
- **Session multiplier:** After clamp, `ApplySessionVolume` multiplies both thresholds by the bucket

entry matching current UTC hour (`session_volume.sessions[]`). Defaults: ASIA 00–07 × 0.80/0.85, LONDON 08–12 × 1.00/1.00, NY 13–23 × 1.15/1.10. Bypassed when `session_volume.enabled = false` or no bucket matches.

Possible ranges: H: 1.6–6.9x. M: 1.2–4.4x. Extremes imply the session is either dead quiet or extremely expansive.

Interpretation:

- `VolumeRatio` ≥ `H` on a candle with price moving in-direction fires a full Volume signal. Below H, above M, and with cross-confirmation fires a mid-tier partial via Pass 2.
- A live H value near the clamp floor (2.0x NY-adjusted, ≈2.3x after session mult) in a quiet session is telling you "relative expansion is cheap here — don't over-weight volume spikes". Conversely a high H (>5x) during quiet intervals means only genuine flushes will register.
- The trader's spec rule is "breakout needs > 3x SMA(9)". The dynamic H replaces the literal 3.0 threshold; when live H drops below 3.0, the engine is relaxing that rule based on session context. Override mentally if you want strict 3x.

1.6.3 VWAP dev thr

What: Dynamic VWAP deviation threshold (as percentage). Displayed with a `(legacy ref)` tag.

Calculation:

- Computed by running a rolling cumulative VWAP over the last `min(50, candles - 1)` samples and collecting `|close - vwap| / vwap × 100` per sample.
- `threshold = clamp(mean(devs) + stdDev(devs), cfg.Indicators.VWAPDynamic.DevClampMin, DevClampMax)` = clamp to `[0.30, 3.0]`.
- Falls back to `StaticFallback` (1.5) if fewer than 10 samples.

Possible values: 0.30–3.0 (%), clamp-bounded.

Interpretation: Marked `(legacy ref)` because the live VWAP scoring uses sigma bands (σ1/σ2) around the dual-session VWAP — this single-dev-% threshold is retained for display/historical compatibility but does not gate the VWAP scoring signal. Read it as a rough intraday volatility-around-VWAP indicator: values near the floor imply tight mean-reversion behaviour; values > 1% imply meaningful one-sided pressure. Not a trading input — use the sigma-band VWAP rows in the scoring breakdown instead.

1.6.4 ATR scale

What: The live ATR scale factor used to stretch/compress ATR-based targets and stops, plus the inputs.

Calculation:

- `ATRRef` — rolling mean of ATR over the recent window (`min(100, candles - period)` rolling ATR values, default ATR period 7). On cold start or insufficient history falls back to `cfg.Indicators.ATR.StaticRef` (115 v14, midpoint of trader-profile Normal band 80–150).
- `ATRScaleFactor = clamp(r.ATR / ATRRef, cfg.Indicators.ATR.ScaleMin, ScaleMax)` = clamp to `[0.25, 4.0]`.
- When `r.ATR = 0` or `ATRRef = 0`, factor defaults to 1.0 and ref falls back to `StaticRef`.

Displayed:

- `scale` — the factor (e.g. 0.79x).
- `ATR` — current raw ATR (price points) from `r.ATR`.
- `ref` — rolling reference ATR or static fallback.

Interpretation:

- `scale > 1.0` → current ATR is above its rolling baseline. The engine widens ATR-based stops and targets proportionally — you're in an expansion regime. Position size down (per trader-profile: low-ATR = larger size, high-ATR = smaller size).
- `scale < 1.0` → current ATR is below baseline. Stops and targets compress, meaning the ATR frame gets tighter. Position size up within risk limits.
- `scale = 1.0` exactly on any non-warm-up run means either the raw ATR matched the reference or the clamp bit — check the raw values.
- Static-fallback `ref = 115.0` vs live `ref = <computed>` is a useful freshness cross-check: if static ref is quoted while `[LIVE]` tag is shown, the live-ref path computed `0` and fell back inside an otherwise-live run (rare, usually means a data gap).

1.7 5. Regime

```
REGIME (5m): TRANSITIONAL
  ADX: 24.3  |  +DI: 27.5  |  -DI: 9.6
```

The regime classifier. Computed **on the 5m timeframe** (not 1m), intentionally — regime is a slower-moving structural read.

1.7.1 Regime label

What: The classifier output, driving the `MaxScore` ceiling, the Step 4 regime veto, and the Pass 2c alignment gate.

Calculation: In `MainForm_Analysis.vb` after `CalcDMI(candles5m, cfg.Indicators.ADX.Period=9)`:

1. **Raw classification:**
   - `ADX > trend_threshold (25)` AND `+DI > -DI` → `TRENDING_UP`
   - `ADX > trend_threshold (25)` AND `-DI > +DI` → `TRENDING_DOWN`
   - `ADX < range_threshold (20)` → `RANGE_BOUND`
   - Else (ADX 20–25) → `TRANSITIONAL`
2. **1-bar hysteresis** (prevents flip-flop on noisy ADX boundary): If `rawRegime = RANGE_BOUND` AND the previous tick was `TRENDING_UP` / `TRENDING_DOWN` / `TRANSITIONAL`, the displayed regime holds `_prevRegime` for one bar. Otherwise the raw classification is used directly. `_prevRegime` updates to the raw value at the end of every run regardless.

Possible values:

Regime	MaxScore (v14 defaults)	Scoring impact
`TRENDING_UP`	20	Step 4 veto: `NO TRADE` if short side wins. Pass 2c TRENDING alignment active.
`TRENDING_DOWN`	20	Step 4 veto: `NO TRADE` if long side wins. Pass 2c TRENDING alignment active.
`RANGE_BOUND`	19	No veto. Pass 2c RANGE_BOUND alignment active (VWAP+RSI+Donchian).
`TRANSITIONAL`	15	Step 4 graded ADX penalty (see below). Pass 2c suppressed.

Base values (`RegimeWeights.Enabled = false`): 19 / 18 / 15. The +1 / +1 / 0 bonuses come from `cfg.RegimeWeights.{Trending,RangeBound}.AlignmentBonus`.

TRANSITIONAL penalty ladder:

ADX band	`RegimePenalty`
`[20.0, 22.5)`	2 (`transitional_penalty_low`)
`[22.5, 25.0)`	1 (`transitional_penalty_mid`)

Applied with a `TierFloor` guard so the raw score cannot penalty-fall below its tier floor (`ls ≥ 12` → `floor 9`, `ls ≥ 9` → `floor 6`, `ls ≥ 6` → `floor 3`, else floor 0).

1.7.2 ADX / +DI / -DI

What: Wilder smoothed ADX and directional indicator values on the 5m series, period 9.

Calculation: `CalcDMI` in `Indicators_Momentum.vb`. Wilder-smoothed true range and directional movement; `+DI = 100 × smoothedDMPlus / smoothedTR`, `-DI = 100 × smoothedDMMinus / smoothedTR`, `DX = 100 × |+DI - -DI| / (+DI + -DI)`, ADX = Wilder-smoothed DX over `period`.

Interpretation:

- The spread `(+DI) - (-DI)` is the directional conviction. In the sample above, 27.5 vs 9.6 is a strong +DI bias — but ADX 24.3 is still sub-trend, so the regime label says `TRANSITIONAL` rather than `TRENDING_UP`. Read this as "direction is clean but the trend hasn't *earned* the label yet".
- Rising ADX through 25 with +DI dominant is the canonical `TRANSITIONAL → TRENDING_UP` transition; watch for it over a few consecutive runs.
- `RANGE_BOUND` after a trend (ADX dipping below 20 following a period above 25) is the regime that triggers the 1-bar hysteresis — meaning one tick of "false range" after a trending sequence will display the previous label. Actual `RANGE_BOUND` only appears after two consecutive qualifying reads.

1.7.3 Colour cue (display only)

Regime line colour: `TRENDING_UP` green, `TRENDING_DOWN` red, `RANGE_BOUND` amber, `TRANSITIONAL` dim grey. No scoring impact — visual at-a-glance state indicator.

1.8 6. Core Signals (1m)

```
CORE SIGNALS (1m):
  ROC(9):       0.115  |  Slope: FLAT
  RSI(9):       52.6  |  Div: BEARISH
  Volume:       0.3701 BTC ($28.1K)  |  vs SMA: 0.16x  |  SMA: 2.3850 BTC
```

Three always-scored primitives on the 1m execution timeframe.

1.8.1 ROC(9)

What: 9-period Rate of Change (percentage) plus a classified slope direction.

Calculation: `CalcROCSeries(candles1m, period=9, lookback=cfg.Indicators.ROC.SeriesLookback=3)`. Builds the last `lookback` ROC values; the displayed `r.ROC` is the most recent. `r.ROCSlope` is classified elsewhere by comparing recent ROC values:

- `RISING` — recent ROC series is trending up AND latest > 0
- `FALLING` — recent ROC series is trending down AND latest < 0
- `FLAT` — otherwise

Scoring use:

- **Full long**: `ROC > 0` AND `ROCSlope = RISING`
- **Full short**: `ROC < 0` AND `ROCSlope = FALLING`
- **Partial long**: `ROC > cfg.Indicators.ROC.SlopeSensitivity` (0.1) AND `ROCSlope != RISING` — can upgrade in Pass 2 with any Momentum cross-confirm.
- **Partial short**: mirrored.
- **Pass 2c TRENDING input**: `active` when `|ROC| ≥ SlopeSensitivity`; then aligned if sign matches dominant side.

Interpretation:

- `ROC` value is read as percent; +0.115 means the 1m close is 0.115% above 9 candles prior.
- `Slope FLAT` with `ROC > 0.1` is the classic "still positive, losing momentum" read — the scoring engine recognises this via the partial path.
- On 1m BTC scalping, `|ROC| > 0.3` is already a strong single-bar impulse; > 0.6 is extension territory and triggers `TAKE PROFIT` hold status if in-position (default `HoldRocTakeProfitLong = 0.6`, `Short = -0.6`).
- ROC crossing zero is a structural exit trigger for open positions (Hold Layer 2).

1.8.2 RSI(9)

What: 9-period Wilder RSI plus optional divergence tag.

Calculation: `CalcRSI(candles1m, 9)`. Wilder EMA-smoothed gain/loss ratio, standard formula. `RSIDivergence` is calculated separately by `CalcRSIDivergence` using **pivot-based** detection (not rolling averages):

- Scans the last `LookbackBars=20` bars for structural swing pivots with `PivotWing=2` confirmation bars on each side.
- `BEARISH` fires when price prints a higher high than the pivot AND RSI at that pivot was lower than current RSI — and both deltas clear `DivergencePriceGate=0.001` and `DivergenceRsiDelta=2.0`.
- `BULLISH` is the mirror for lower-low price pivots.
- Returns `NONE` otherwise.

Scoring use:

- **Full long**: `RSI > overbought (60)`
- **Full short**: `RSI < oversold (40)`
- **Partial long**: `RSI > partial_overbought (55)` AND `RSI` $\leq$ `60` — **dead band 45–55** (v14). Can upgrade in Pass 2.
- **Partial short**: `RSI < partial_oversold (45)` AND `RSI` $\geq$ `40`.
- **Divergence penalty**: `BEARISH` AND `RSI > DivPenaltyRsiHigh (65)` → `-1 [L]`. `BULLISH` AND `RSI < DivPenaltyRsiLow (35)` → `-1 [S]`.
- **Pass 2c RANGE_BOUND input**: `active` always; aligned if `RSI > 50` (for long side) or `RSI < 50` (short).

Display variants:

- `Div:` line **appended only** when `RSIDivergence != NONE` and not empty. No line means no structural divergence detected in the lookback window.
- Colour: red above 70, green below 30, grey between.

Interpretation:

- The 45–55 dead band is deliberate — RSI near the neutral line provides no directional edge and used to double-count momentum via the old `50/50` partial thresholds. Post-v14, you'll see RSI rows without `[L*]` / `[S*]` hits for every small drift off 50.
- A `Div: BEARISH` tag with `RSI < 65` is visible but does **not** penalise scoring — the penalty gate is deliberately set above the overbought line to filter noise. If you see `Div: BEARISH` with RSI 58–64, it's a soft warning to tighten exits, nothing more.
- RSI is the hold manager, not an entry trigger — per trader profile, act on RSI during trade management (`> 60` hold, `< 40` exit for longs), not for entry.

1.8.3 Volume

What: Current 1m candle volume (BTC), USD notional, ratio to 9-period SMA, and the SMA value itself.

Calculation:

- `CurrentVolume` — `candles1m.Last().Volume` in BTC.
- `CurrentVolumeUSD` — computed elsewhere in `MainForm_Analysis` from current close $\times$ volume. Formatted with `M` / `K` / plain suffix per magnitude.
- `VolumeSMA9` — `CalcVolumeSMA(candles1m, 9)` over last 9 candles.
- `VolumeRatio` — `CurrentVolume / VolumeSMA9`.

Scoring use (with Dynamic Norms thresholds H and M):

- **Full long**: `VolumeRatio` $\geq$ `H` AND `ROC > 0` AND `CurrentPrice > VWAP`
- **Full short**: `VolumeRatio` $\geq$ `H` AND `ROC < 0` AND `CurrentPrice < VWAP`
- **Partial (mid-tier)**: `VolumeRatio` $\geq$ `M` AND `VolumeRatio < H`. Upgrades to full score in Pass 2 only with a Volume-category cross-confirm (OBV).
- No score fires when `VolumeRatio < M`.

Display colour: green $\geq$ 1.5x, red < 0.7x, grey otherwise.

Interpretation:

- The USD column is a sanity check against the BTC ratio — 0.16x may sound dead but if BTC is trading at $76k, the actual notional still matters. On 1m BTC perp, `> $500K` notional usually means real participation regardless of the ratio.
- A `VolumeRatio < 0.7x` (red) during price movement is a fade warning — price moving without volume rarely sustains. The engine doesn't penalise low volume directly, but the full/partial volume tiers won't fire, so directional signals lose a line of confirmation.
- Remember: the trader-profile 3x rule is subsumed by the Dynamic Norms H threshold. A `0.16x` ratio isn't even close to any tier; a `2.5x` ratio in NY session (H ≈ 4x) still won't fire full volume but may qualify mid-tier.

1.9 7. VWAP

```
VWAP (reset 13:30 UTC):
  Value:  75995.6  |  Dev: 0.056%  |  Candles: 190
  s1 band: [75715.9, 76275.3]  |  s2 band: [75436.2, 76555.0]
```

Dual-session auto-anchored VWAP with volume-weighted sigma bands.

1.9.1 Section header

Format: `VWAP (reset HH:MM UTC)<[WARMUP]>`

- `HH:MM` — whichever session anchor is currently active. Anchors at `Session1StartHour:Minute` (default `00:00 UTC`) and `Session2StartHour:Minute` (default `13:30 UTC`). The displayed anchor is the most recent one (the one that reset the VWAP accumulator for the current session).
- **`[WARMUP]` tag** appears when `r.VWAPSessionCandles < cfg.Indicators.VWAP.WarmupCandles (15)` — i.e. fewer than 15 candles since the last session anchor.

Warmup behaviour: Scoring of VWAP full/partial is **suppressed** during warmup. The breakdown row still shows the warmup message instead of a signal hit. Cross-category upgrades

don't fire off VWAP during this window.

Interpretation: The `[WARMUP]` tag coming back ~15 minutes after 00:00 or 13:30 UTC is expected behaviour — don't trust VWAP-based scoring rows during that window. Outside warmup, the anchor in the header tells you which session's VWAP is live: `00:00 UTC` anchor dominates Asian/European hours, `13:30 UTC` anchor dominates US session.

1.9.2 Value / Dev / Candles

What:

- `Value` — session VWAP (volume-weighted typical price cumulated since anchor).
- `Dev` — `(CurrentPrice - VWAP) / VWAP × 100` as percent. Signed (positive = price above VWAP).
- `Candles` — `VWAPSessionCandles`, count of 1m candles since the session anchor.

Calculation (`CalcVWAP` in `Indicators_Volatility.vb`):

- Anchor selection: if `UtcNow` is before `Session2` (default `13:30 UTC`), anchor = start of UTC day (`00:00`). Else anchor = `13:30 UTC` today.
- Accumulate `typicalPrice × volume` and `volume` from anchor onwards where `typicalPrice = (H + L + C) / 3`.
- `VWAP = cumTPV / cumVol`.

Display colour: Dev shown amber when `|Dev| > norms.VWAPDevThreshold`; grey otherwise. (As noted in Dynamic Norms, the threshold is a `legacy ref` — scoring uses sigma bands, not Dev%.)

Interpretation:

- Dev reads as session "air gap" above/below fair value. On BTC perp, `|Dev| > 0.3%` is meaningful; > 0.5% is extended and often mean-reverts.
- `Candles` count lets you verify session integrity — after 13:30 UTC reset you should see it drop back to low single digits then climb. A stuck count (e.g. 200+ after what should have been a reset) means something broke in session detection.

1.9.3 s1 band / s2 band

What: 1-sigma and 2-sigma **volume-weighted** bands around VWAP. Not percentile bands — true weighted standard deviation of `typicalPrice` from VWAP.

Calculation (`CalcVWAPBands`):

- Over the same session candles, accumulate `volume × (typicalPrice - VWAP)²` and `volume`.
- `sigma = sqrt(cumWeightedSqDev / cumVol)`.
- `s1 = VWAP ± sigma`, `s2 = VWAP ± 2σ`.

Scoring use:

- **Full long (Microstructure category)**: `VWAP < Price ≤ s1Upper` — price is above VWAP but still within the first band.
- **Full short**: `s1Lower ≤ Price < VWAP`.
- **Partial long** (can upgrade via Pass 2 with any Microstructure cross-confirm): `s1Upper < Price ≤ s2Upper`.
- **Partial short**: `s2Lower ≤ Price < s1Lower`.
- No score above `s2Upper` or below `s2Lower` — price is too extended for a fresh-direction signal.
- **Pass 2c RANGE_BOUND input**: `active` only when not in warmup; then aligned if price above VWAP (long) or below (short).

Interpretation:

- Price sitting just above VWAP with `Dev < 0.1%` typically means the s1 band contains it (full long score). This is the "clean mean-reversion long" posture.
- Between s1 and s2 is "leaning extended" — the engine requires cross-confirmation before awarding the score, hence the partial-upgrade gate.
- Beyond s2 is where you'd look for **short against** the extension (if you're a mean-reverter) or **cut exposure** (if you're running with the trend). The engine goes silent either way — by design, it doesn't pick a side in stretched tails.
- A price in the middle of wide s1 / s2 bands implies high realised intraday volatility; tight bands imply consolidation. The width is itself information, even though not scored directly.

1.10 8. BBW / TTM Squeeze

```
BBW / TTM SQUEEZE:
  BBW: 0.004  |  Status: NONE
  TTM: Histogram=70.26  Dir=FALLING  Signal=BULL_FADING
```

Bollinger Band Width (compression detector) plus TTM Squeeze Momentum (directional momentum oscillator).

1.10.1 BBW

What: Bollinger Band Width (normalised), and a classified squeeze status.

Calculation (`CalcBBW` in `Indicators_Volatility.vb`):

- Rolling 20-period Bollinger Band over the last `period * 5 = 100` candles.

- `BBW = (upper - lower) / mid`, where bands are `mid ± stdMult × stdDev` and `mid` is the 20-period SMA of closes.
- `r.BBW` = most recent value in the series.

Squeeze threshold (v0.48):

- `threshold = 20th percentile` of the rolling BBW series (not `min × 1.5`).
- `SqueezeStatus`:
  - `ACTIVE` — current BBW $\leq$ threshold (in the bottom 20% of recent volatility)
  - `RELEASING` — previous BBW was $\leq$ threshold, current is above (just broke out of squeeze)
  - `NONE` — neither

Possible values: BBW typically 0.001–0.050 on 1m BTC (normalised). Status one of the three strings above.

1.10.2 TTM

What: Linear-regression momentum histogram, its slope direction, and a four-state signal combining the two.

Calculation (`CalcTTMSqueeze`):

- For each of the last `linRegPeriod=7` candles, compute `close - SMA20(close)`. This is the histogram input series.
- Fit a least-squares linear regression through these 7 values; `Histogram = intercept + slope × (n-1)` — the regression's rightmost (latest) projected value.
- `Direction` classified by comparing first-third-mean vs last-third-mean of the delta series:
  - `delta > flat_threshold (0.5)` → `RISING`
  - `delta < -flat_threshold` → `FALLING`
  - Else → `FLAT`
- `Signal` cross-tabulated:

Histogram sign	Direction	Signal
`> 0`	`RISING`	`BULL_BUILDING`
`> 0`	`FALLING`	`BULL_FADING`
`< 0`	`FALLING`	`BEAR_BUILDING`
`< 0`	`RISING`	`BEAR_FADING`
any	`FLAT`	`FLAT`

1.10.3 Combined scoring logic (BBW × TTM)

Implemented in `ScoringEngine_Calculate_Scoring.vb` as a single Select on `SqueezeStatus`:

`SqueezeStatus`	TTM Signal	Scoring effect
`ACTIVE`	any	`LongScore -= BbwSqueezePenalty (2)` AND `ShortScore -= 2`. Both sides penalised during compression.
`RELEASING` / `NONE`	`BULL_BUILDING`	`LongScore += 1` (Microstructure category). Shown as `[L]` in breakdown.
`RELEASING` / `NONE`	`BEAR_BUILDING`	`ShortScore += 1` (Microstructure). Shown as `[S]`.
`RELEASING` / `NONE`	`BULL_FADING` / `BEAR_FADING` / `FLAT`	No score change, no penalty — breakdown row reads `-- no award`.

1.10.4 Display

- **BBW line** — value (3-decimal) and status. Colour is an intended `C_WARN` when status is `SQUEEZE`, but the code checks the literal string `"SQUEEZE"` which is never emitted — the actual emitted values are `ACTIVE`/`RELEASING`/`NONE`, so the BBW line always renders grey. (Cosmetic no-op; scoring is unaffected.)
- **TTM line** — histogram (2-decimal), direction, signal. Colour intended to be green/red on direction `UP`/`DOWN`, but the code emits `RISING`/`FALLING`, so the TTM line also always renders grey. (Same dead colour path.)

1.10.5 Interpretation

- **`ACTIVE` squeeze** is the engine's compression detector — both sides are penalised because direction is undefined during compression. Don't force entries into an ACTIVE squeeze; wait for the release.
- **`RELEASING` status + `BULL_BUILDING` signal** is the highest-quality compression-to-expansion long trigger the engine expresses. In the scoring table this shows as a single `[L]` on the `BBW/TTM` row, but its predictive value is higher than a generic +1 — volatility expansion *with direction* is the core setup the trader profile wants to catch.
- **`BULL_FADING` / `BEAR_FADING`** are inflection warnings: histogram is still on one side of zero, but direction is rolling over. The engine awards nothing, but the `CONTEXT: MOMENTUM_FADING` tag

uses `TTMSignal ∈ {BULL_FADING, BEAR_FADING}` as one of its three fading-count inputs. So even though TTM didn't score, it can still flip the context tag.

- **Histogram magnitude** (unscored) is worth eyeballing: `|Histogram| < 5` on a `FADING` signal means the momentum wave is almost exhausted. Large histograms (`|Histogram| > 50`) with `BUILDING` direction means a real directional wave is under construction.
- The 20th-percentile squeeze threshold means `ACTIVE` fires genuinely into the tail of recent realised volatility — not on any session-low spike. Expect it to appear during genuine consolidation phases, typically between two trending segments.

1.11 9. EMA Ribbon

```
EMA RIBBON (1m):
  9: 76048.6  |  21: 76007.3  |  50: 75946.6  |  Align: BULL
  5m EMA200: 76040.4  |  Price: BELOW
```

Two-layer EMA view: 9/21/50 ribbon on 1m as the dynamic trend structure, plus 5m EMA(200) as the higher-timeframe anchor.

1.11.1 EMA 9 / 21 / 50 (1m)

What: Three exponential moving averages on the 1m execution timeframe and their alignment label.

Calculation: `CalcEMA(candles1m, period)` in `Indicators_Momentum.vb`. Standard EMA: seed = SMA of first `period` closes, then `ema = close × k + ema × (1 − k)` where `k = 2 / (period + 1)`. Periods 9, 21, 50 sourced from `cfg.Indicators.EMA.{Fast,Mid,Slow}`.

`EMAAlignment` is classified by strict ordering:

Condition	Alignment
`EMA9 > EMA21 > EMA50`	`BULL`
`EMA9 < EMA21 < EMA50`	`BEAR`
Anything else	`MIXED`

Scoring use:

- **Full long (MarketStructure category)**: `EMAAlignment = BULL` → `+1 LongScore`.
- **Full short**: `EMAAlignment = BEAR` → `+1 ShortScore`.
- **MIXED**: no score.
- **Pass 2c TRENDING input**: always active; aligned if alignment matches dominant side.

- **Proposed direction for MTF Gate**: `BULL` proposes LONG, `BEAR` proposes SHORT (if regime is also not countering) — feeds into `CalcMTFGate` pre-check.

Display colour: green on `BULL`, red on `BEAR`, amber on `MIXED`.

Interpretation:

- The ribbon is the most influential single scoring primitive in TRENDING regimes — both a direct +1 and a Pass 2c alignment vote.
- `MIXED` is a legitimate "standing-aside" signal. The most common cause is EMA9 crossing but EMA21 still ahead of EMA50 (or vice versa) — a ribbon in transition. No vote until the three lines reclaim monotonic order.
- Ribbon spread (EMA9 − EMA50) implies trend strength. A `BULL` alignment where the three EMAs are within a few dollars of each other is a weak ribbon about to churn into `MIXED`; a wide spread is a mature trend.
- The ribbon is slow — it will mis-fire during sharp reversals that haven't yet propagated into EMA50. Cross-check against EMA200(5m) for whether the reversal is local noise or a higher-timeframe event.

1.11.2 5m EMA(200)

What: 200-period EMA on the 5m candles; regime anchor.

Calculation: `CalcEMA(candles5m, 200)`. Requires $\geq$ 200 candles of 5m history (~17 hours). Returns 0 when insufficient data — `PriceVsEMA200` shows `N/A` in that case.

`PriceVsEMA200` classification:

Condition	Label
`CurrentPrice > EMA200_5m` (and `EMA200_5m > 0`)	`ABOVE`
`CurrentPrice < EMA200_5m` (and `EMA200_5m > 0`)	`BELOW`
`EMA200_5m = 0`	`N/A`

Scoring use:

- **Full long (MarketStructure)**: `ABOVE` → `+1 LongScore`.
- **Full short**: `BELOW` → `+1 ShortScore`.
- **N/A**: no score (cold start).

Display colour: green `ABOVE`, red `BELOW`.

Interpretation:

- This is a one-way regime veto in practice — price below the 5m EMA200 tilts the scoring towards short, above towards long, regardless of short-term 1m structure. The sample output shows the common conflict case: 1m ribbon `BULL` (+1 Long) but 5m EMA200 `BELOW` (+1 Short). That's the engine expressing "local long, higher-TF short" — healthy disagreement that gets resolved by the rest of the scoring stack.
- A price within a few dollars of EMA200 (like the sample's `76038 vs 76040`) is structurally on the line — a single 1m candle could flip the classification. Treat borderline EMA200 reads as noise; wait for decisive separation.
- `N/A` appearing after session start usually means the 5m candle fetch didn't return enough history. Engine is flying with one side of the scoring disabled. Re-run once data's caught up.

1.12 10. Market Structure

```
MARKET STRUCTURE:
  Donchian(20): Upper=76210.5  Lower=75868.0  |  Signal: NONE
  OBV: Trend=RISING  |  Div=BULLISH
  VPFR-lite: POC:75911.6  |  NEAR_HVN_RESIST  |  HVN@POC:YES
```

Three structural primitives: breakout levels (Donchian), volume-trend confirmation (OBV), and session volume profile (VPFR-lite).

1.12.1 Donchian(20)

What: 20-period high/low channel plus a quartile-aware breakout signal.

Calculation: `CalcDonchian` returns `Upper = max(high)` / `Lower = min(low)` over the last 20 candles. `DonchianSignal` is set downstream (not in `CalcDonchian`) by comparing `CurrentPrice` to the channel and its quartiles:

- Price at or above `Upper` → `LONG`
- Price in upper quartile (below Upper, above Upper − 0.25 × range) → `LONG_PARTIAL`
- Price at or below `Lower` → `SHORT`
- Price in lower quartile (above Lower, below Lower + 0.25 × range) → `SHORT_PARTIAL`
- Mid-channel (middle half) → `NONE`

Scoring use:

- **Full long / short (MarketStructure)**: `LONG` / `SHORT` → `+1`.
- **Partial**: `LONG_PARTIAL` / `SHORT_PARTIAL` — can upgrade in Pass 2 with any MarketStructure cross-confirm.

- **NONE**: no score, no partial. Breakdown row note reads `MID-CHANNEL -- no signal`.
- **Pass 2c RANGE_BOUND input**: aligned if signal matches dominant side (accepts both LONG and LONG_PARTIAL as "long-aligned", same for short).

Display colour: green on `LONG`, red on `SHORT`, grey otherwise (partial signals display grey; the `[L]` / `[S]` hit still shows in breakdown).

Interpretation:

- Full `LONG` / `SHORT` = price just printed a 20-minute extreme. Genuine breakout that warrants cross-confirm from order flow and volume before acting.
- `LONG_PARTIAL` (upper quartile) is "pressing the highs" — the right bid environment for a breakout but the level hasn't broken yet. The partial-upgrade path makes this score only when other MarketStructure signals (EMA ribbon, DMI, ADX, 5m EMA200, VPFR) agree.
- `NONE` in the middle half is silent by design — mid-channel price has no breakout edge. Don't read absence as disconfirmation.
- Width of the channel (`Upper - Lower`) is an implicit volatility proxy separate from ATR; use it to sanity-check the ATR-based stop distances.

1.12.2 OBV

What: On-Balance Volume trend direction plus price/OBV divergence state.

Calculation (`CalcOBV` in `Indicators_Structure.vb`):

- OBV accumulated across all supplied candles: `+volume` on up-close, `-volume` on down-close, no change on flat.
- `obvChange = (obvLast - obvFirst) / |obvFirst|` if non-zero, else 0.
- `OBVTrend`:
  - `obvChange > trend_gate (0.001)` → `RISING`
  - `obvChange < -trend_gate` → `FALLING`
  - Else → `FLAT`
- `OBVDivergence` (only when `|priceChange| ≥ divergence_gate (0.001)`):
  - Price up + OBV down → `BEARISH`
  - Price down + OBV up → `BULLISH`
  - Else / price change below gate → `NONE`

Scoring use:

- **Full long (Volume category)**: `OBVTrend = RISING` AND `OBVDivergence != BEARISH` → `+1`.

- **Full short**: `OBVTrend = FALLING` AND `OBVDivergence != BULLISH` → `+1`.
- **Partial long**: `OBVTrend = RISING` AND `OBVDivergence = BEARISH` — **adverse divergence blocks cross-category upgrade.** The partial does NOT promote to full in Pass 2 even with a Volume cross-confirm. Breakdown row appends `[upgrade blocked]`.
- **Partial short**: mirrored (`FALLING` + `BULLISH` blocks upgrade).

Display colour: always grey (single colour path — no conditional). Signal state is legible from the `Trend=` and `Div=` strings.

Interpretation:

- OBV aligned with its own trend (no divergence) is a clean Volume-category confirm — the +1 is secondary to the Volume indicator itself but still contributes to any Pass 2 upgrade targeting the Volume category.
- The divergence-blocks-upgrade gate exists for a reason documented in v0.42: OBV disagreeing with its own price trend means volume is not backing the move. Awarding the upgrade would reward a weak signal. Expect to see `Trend:RISING Div:BEARISH | [upgrade blocked]` during late-stage rallies where new highs are printed on fading participation.
- In the sample, `Trend=RISING Div=BULLISH` — trend and divergence pointing the same direction (bullish tilt). This fires `obvLong` normally and does NOT block upgrades.

1.12.3 VPFR-lite v2 (POC + VAH/VAL + nearest HVN/LVN)

What: Volume Profile Fixed Range over the available 1m window, with exponential decay weighting toward recent bars. Reports POC price, value-area boundaries (VAH / VAL), nearest HVN/LVN walls above and below, HVN-near-POC flag, and a signal classification.

v2 additions over v1: value-area boundaries (`VPFRVAH` / `VPFRVAL`) capture the 70%-volume zone; `VPFRNearestHvnAbove` / `VPFRNearestHvnBelow` are the closest high-volume walls in each direction. v2 fields drive the Step 5b 3-tier target cap (swing → nearest HVN → POC).

v1 (legacy): Volume Profile Fixed Range over the available 1m window, with exponential decay weighting toward recent bars. Reports POC price, HVN-near-POC flag, and a signal classification.

Calculation (`CalcVPFRLite` in `Indicators_Structure.vb`):

- Price range: `[min(low), max(high)]` across supplied candles. Split into `numBuckets=50` equal-width buckets.
- Volume per bucket: each candle contributes `volume × decayBase^age` where `decayBase=0.985` and `age = (candleCount - 1 - candleIndex)`. Most recent candle has weight 1.0; weight falls ~22% per 15 bars.

- `POC` = midpoint of bucket with the highest accumulated weighted volume.
- `HVNNearPoc = |CurrentPrice - POC| / POC ≤ hvnProximityPct (0.002)` — i.e. within ±0.2% of POC.
- HVN threshold: `pocVol × hvnVolPct (0.6)`. LVN threshold: `pocVol × lvnVolPct (0.2)`.
- Current bucket's weighted volume → `curBucketVol`.

`VPFRSignal` classification:

Condition	Signal
`HVNNearPoc` AND `CurrentPrice < POC`	`NEAR_HVN_SUPPORT`
`HVNNearPoc` AND `CurrentPrice ≥ POC`	`NEAR_HVN_RESIST`
`!HVNNearPoc` AND `curBucketVol ≤ LVN threshold` AND `CurrentPrice > POC`	`IN_LVN_BULL`
`!HVNNearPoc` AND `curBucketVol ≤ LVN threshold` AND `CurrentPrice < POC`	`IN_LVN_BEAR`
Otherwise	`NEUTRAL`

Scoring use:

- **Full long (MarketStructure)**: `NEAR_HVN_SUPPORT` OR `IN_LVN_BULL` → `+1 LongScore`.
- **Full short**: `NEAR_HVN_RESIST` OR `IN_LVN_BEAR` → `+1 ShortScore`.
- **`NEUTRAL`**: no score.
- **Step 5b ATR target cap** (separate from scoring): when `NEAR_HVN_RESIST` or `IN_LVN_BEAR` AND `POC > CurrentPrice` AND `POC < rawLongTarget` → long target capped at POC. Mirrored for `NEAR_HVN_SUPPORT` / `IN_LVN_BULL` on short side.

Display colour: green for support / LVN-bull, red for resist / LVN-bear, dim for neutral. `HVN@POC` shows `YES` / `NO`.

Interpretation:

- `NEAR_HVN_RESIST` reads as "price is sitting just below a high-volume node acting as resistance" — structurally bearish. If you see it on a breakout attempt, expect rejection. The engine turns this into a short signal + a long-target cap.
- `IN_LVN_BULL` / `IN_LVN_BEAR` fire when price is through a low-volume node — these are the "price moves fast through empty air" zones. Directional score only fires when price is on the

"correct" side of POC relative to the vacuum. A NEUTRAL signal can mean either mid-profile price or normal-density bucket; read it as "no structural information" rather than "balanced".

- POC itself is a useful reference even when the signal is NEUTRAL — it's the volume centre of gravity for the current session window. Traders often watch POC cross as a separate context cue.
- Exponential decay means POC will drift with recent participation rather than anchor on early-session high-volume prints. If you see POC shifting alongside price through the session, that's normal; if POC is static while price diverges, older volume is dominating — the signal is lagging actual structure.
- **VAH/VAL** define the 70%-volume zone. Price inside the value area is "accepted"; price outside is "rejected" or "extended". v2's value-area boundaries don't currently feed scoring directly, but VPFRVAH / VPFRVAL are logged to CSV columns 72/73 for later auto-tweaker tuning.
- **Nearest HVN walls** (VPFRNearestHvnAbove / VPFRNearestHvnBelow) are the closest high-volume nodes in each direction. They feed the Step 5b 3-tier target cap as the second-priority cap (after swing target, before POC).

1.12.4 Swing Pivots (5m + 15m)

What: Most recent confirmed swing high and swing low on 5m and 15m candles, plus direction-aware target/stop bookkeeping for the long and short side.

Calculation (`CalcSwingPivots` in `Indicators_Structure.vb`):

- Walk backward from `scanEnd = candleCount - pivotWing - 1` to `scanStart = pivotWing` looking for confirmed pivots.
- A bar at index `i` is a confirmed swing high iff `high[i] > high[i±k]` for all `k ∈ [1, pivotWing]` (strict inequality on both sides — equal-high bars don't count).
- Mirror logic for swing low.
- First match wins — emits the most recent confirmed pivot, not the highest/lowest in the window.
- 5m: `pivotWing = cfg.Indicators.Swing.PivotWing5m` (default 3), `lookback = LookbackBars5m` (default 30).
- 15m: separate config keys (`PivotWing15m`, `LookbackBars15m`) — narrower wing acceptable due to slower bar pace.

Direction-aware bookkeeping (computed inline in `MainForm_Analysis.RunAnalysisAsync`, not in `CalcSwingPivots`):

- `SwingTargetLong = LastSwingHigh5m` if it's above current price, else `0`.
- `SwingStopLong = LastSwingLow5m` if it's below current price, else `0`.
- Mirror for short side.

Scoring use:

- **Step 5b 3-tier cap (top priority):** when a long verdict has `SwingTargetLong > 0` and `SwingTargetLong < rawLongTarget`, the target is capped to the swing high. Cap is closest-wins across (swing → nearest HVN → POC) — the tightest candidate wins. `TargetCapReason` records which tier won.
- **CalcHoldStatus Layer 1.5 (structural break exit):** during a long position, if `CurrentPrice < SwingStopLong - 0.5 × cfg.Indicators.Structure.SwingBreachAtrMult × ATR`, fires `EXIT -- structural break (swing low breach)`. Sits between Layer 1 (microstructure exit) and Layer 2 (OBV divergence).
- **VerdictContext STRUCTURALLY_WEAK:** when `LastSwingHigh5m > 0 OR LastSwingLow5m > 0` AND no clean target+stop pair can be placed for the verdict direction, fires the tag. Catches the "we have structure but the trade doesn't have R:R" case.

Display: Renders under `MARKET STRUCTURE` as `Last Swing High 5m: 102450.0 | Last Swing Low 5m: 101800.0` with corresponding 15m row dim. ATR Entry block includes a structural row showing `LONG Stop: <swing low> Entry: <price> Target: <swing high> R:R <ratio>` in cyan when both sides exist.

Interpretation:

- Confirmed pivots avoid equal-high false positives. A new swing high at the same level as the prior one does not register, which is the right behaviour — true breakout structure requires a strictly higher high.
- The walk-backward scan picks the most-recent pivot rather than scanning forward from the start. This means the engine always references the freshest actionable level, not the oldest historical reference in the lookback.
- 15m pivots are display-only context. They are not used for cap arbitration or structural break detection — only the 5m levels drive those.
- A `STRUCTURALLY_WEAK` tag with `0,0` in `SwingTargetLong/SwingStopLong` is the diagnostic case: enough candle history exists for at least one swing detection, but the geometry doesn't produce a clean R:R for the verdict direction.

1.12.5 Trend Structure (HH/HL/LH/LL)

What: Sequence-of-pivots classification on the 5m timeframe — UPTREND / DOWNTREND / EXPANSION / CONTRACTION / UNDEFINED.

Calculation (`ClassifyTrendStructure` in `Indicators_Structure.vb`):

- Walk backward through `candles5m` to identify the last `cfg.Indicators.TrendStructure.PivotCount` (default 6) confirmed pivots, each requiring `cfg.Indicators.TrendStructure.PivotWing` (default 3) confirmation bars on each side. Mix of highs and lows.
- Need at least 2 highs AND 2 lows in the result. If fewer (window too short or chop), return `UNDEFINED`.
- Compare the most recent two highs: `HH` if newer > older; `LH` otherwise.
- Compare the most recent two lows: `HL` if newer > older; `LL` otherwise.
- Map to enum:
  - `HH + HL` → `UPTREND`
  - `LH + LL` → `DOWNTREND`
  - `HH + LL` → `EXPANSION`
  - `LH + HL` → `CONTRACTION`

Pure function. Returned as `r.TrendStructure`. Two display tuples (`LastTwoHighs5m`, `LastTwoLows5m`) carry the four prices used in the comparison.

Scoring use (Pass 2c, after TRENDING / RANGE_BOUND alignment scoring, before snapshot for funding modifiers):

```
If cfg.RegimeWeights.Enabled AND cfg.Indicators.TrendStructure.Enabled:
   UPTREND    AND LongScore > ShortScore  → LongScore += structure_bonus  (capped at regimeMax)
   DOWNTREND  AND ShortScore > LongScore  → ShortScore += structure_bonus (capped at regimeMax)
    EXPANSION   → no score change (display caution flag)
    CONTRACTION → no score change (range-narrowing context)
    UNDEFINED   → no score change
```

`structure_bonus = cfg.Indicators.TrendStructure.StructureBonus` (default 1). Suppressed in TRANSITIONAL regime (consistent with rest of Pass 2c). Bonus only applies when structure agrees with the dominant side — never both, never opposite.

Display: Renders under `MARKET STRUCTURE` as:

```
Trend Structure : UPTREND  (HH 102450.0 > 102100.0 | HL 101800.0 > 101500.0)
```

Colours: UPTREND green, DOWNTREND red, EXPANSION amber, CONTRACTION dim cyan, UNDEFINED dim grey.

CSV column 87: `TrendStructure5m` — string enum value. Logged per row.

Interpretation:

- Trend Structure is structure-on-pivots, distinct from EMA / ROC / CVD which are price-momentum signals. The Pass 2c bonus is deliberately separate from the existing regime-alignment bonus to avoid double-counting.
- `EXPANSION` is the most informative non-scoring state — both higher highs AND lower lows indicate the market is widening its range. Often precedes a regime shift or volatile rejection of a recent breakout. Treat as a caution flag for fresh entries.
- `CONTRACTION` is range-narrowing, often paired with low BBW. The next decisive move is usually directional but the structure alone doesn't tell you which way.
- `UNDEFINED` typically appears in the first ~30 minutes of a session before enough confirmed pivots accumulate, or in deep chop where the wing-strict-inequality test fails. Don't read it as bearish or bullish.

1.12.6 Best Volume Pivot (Display-Only)

What: The pivot in the 5m lookback with the highest total wing-window volume, reported alongside its volume ratio against the average pivot in the same lookback. Display-only in v1 — no scoring or cap-arbitration impact.

Calculation (extension of `CalcSwingPivots`):

- For each confirmed pivot found in the lookback, sum the volume of all bars in `[pivotIdx - pivotWing, pivotIdx + pivotWing]` — total volume across the wing window.
- Track the pivot with the highest total wing-window volume (`BestPivotByVolume5m`).
- Compute average wing-window volume across all pivots in lookback (`avgPivotVolume`).
- `BestPivotVolumeRatio5m = best / avgPivotVolume`.
- `BestPivotIsHigh5m` = True if best pivot is a swing high; False if a low.
- If fewer than 2 confirmed pivots in lookback, all three fields return `0` / `0` / `False`.

Scoring use: none in v1. Logged for offline analysis and CalibrationReport `BEST VOLUME PIVOT DISTRIBUTION` section.

v2 promotion condition (parked in `DeribitIndicatorProject.md` §16.6 P1): if CalibrationReport shows "best is also most-recent" rate falls below 50% AND auto-tweaker output shows volume-weighted pivots correlate with subsequent target-hit rate, promote to a 4th cap tier (best-volume-swing > most-recent-swing > nearest HVN > POC).

Display: Renders under `MARKET STRUCTURE` below the swing pivot rows:

```
Best Vol Pivot 5m: HIGH 102450.0  (vol×2.3 vs avg pivot)
```

Colour: dim cyan (informational, not actionable in v1).

CSV columns 85–86: `BestPivotByVolume5m` (price) and `BestPivotVolumeRatio5m` (ratio). Reserved in v0.4 schema; populated when D2 ships.

Interpretation:

- Ratios above 2.0× indicate a meaningfully stronger reference level than the average pivot. Under 1.5× the volume-weighting is barely differentiating from the most-recent pivot.
- When `BestPivotByVolume5m` differs from `LastSwingHigh5m` (or low), there's a stronger pivot back in the lookback than the most-recent one. Eyeball whether your structural target/stop should reference it instead.
- This is a chart-reading aid that exposes the volume-quality of the pivots being used elsewhere — not a fresh trade signal.

1.13 11. Open Interest

```
OPEN INTEREST:
  OI: 1036558700  |  d15m: 0.000%  |  d60m: 0.000%  |  Signal: NEUTRAL
```

Aggregate BTC-PERPETUAL open interest plus 15m / 60m percentage changes and a directional classifier.

1.13.1 OI / d15m / d60m

What:

- `OI` — current absolute OI value from Deribit `get_book_summary_by_instrument` (`r.OI_Current`).
- `d15m` — percentage change of OI vs the snapshot ~15 minutes ago.
- `d60m` — percentage change vs ~60 minutes ago.

Calculation: In `MainForm_Analysis.vb` using an `OiSnapshot` ring buffer keyed by timestamp (`_oiHistory`). Each run appends `(nowTs, OI_Current)`. When computing deltas, looks up the stored snapshot closest to `nowTs - 15m` / `nowTs - 60m`. If no matching snapshot exists (cold start / recent restart), delta reads `0.000%`.

Interpretation:

- On a fresh session, d15m and d60m both read `0.000%` for the first ~15 and ~60 minutes

respectively — this is warmup, not a real flat OI reading. The signal classifier will hold at `NEUTRAL` during this window.

- Absolute `OI` value is useful for magnitude context but doesn't directly score — the engine only reads the deltas.
- BTC-PERPETUAL OI typically swings ±0.1% to ±1.0% over 15 minutes in active sessions. Anything beyond ±1% in 15m is a meaningful positioning event worth eyeballing.

1.13.2 Signal

What: Directional classification of the OI change in combination with price direction.

Calculation: Derived from `d15m` / `d60m` against `cfg.Indicators.OI.{NeutralBandPct (0.05), ChangeThresholdPct (0.01)}` cross-referenced with recent price direction. Produces one of five states.

Possible values:

Signal	Meaning	Condition (approximate)
`NEW LONGS`	Fresh long positioning — OI up + price up	OI rising past threshold AND price rising
`NEW SHORTS`	Fresh short positioning — OI up + price down	OI rising past threshold AND price falling
`COVERING`	Shorts covering — OI down + price up	OI falling past threshold AND price rising
`CAPITULATION`	Longs capitulating — OI down + price down	OI falling past threshold AND price falling
`NEUTRAL`	OI change within neutral band OR during warmup	Default / no qualifying move

Scoring use:

- **Full long (Microstructure)**: `NEW LONGS` → `+1 LongScore`.
- **Full short**: `NEW SHORTS` → `+1 ShortScore`.
- **Partial long**: `COVERING` — upgradeable in Pass 2 with any Microstructure cross-confirm.
- **Partial short**: `CAPITULATION` — upgradeable similarly.
- **Pass 2b OI × CVD cross-confirm**: Full `NEW LONGS` + bullish CVD slope & value → `+UpgradeBonus (1)` long. Full + opposing CVD → `-ConflictPenalty (1)` long. Mirror for `NEW SHORTS`. Upgraded `COVERING` / `CAPITULATION` (after Pass 2 upgrade) can *confirm* but cannot trigger a conflict penalty — by design, to avoid double-penalising short-covering or capitulation transitions.

Display colour:

- Green: `NEW LONGS`, `COVERING`
- Red: `NEW SHORTS`, `CAPITULATION`
- Grey: `NEUTRAL`

Interpretation:

- `NEW LONGS + CVD RISING` is the highest-quality Pass 2b confirmation — genuine new long participation, confirmed by aggressor flow. `+1` raw score + `+1` Pass 2b bonus.
- `NEW LONGS` with `CVD FALLING` is the conflict case — OI building but sell-side aggression. Often means leveraged positions being built into selling, which historically unwinds violently. `-1` Pass 2b penalty is deliberate.
- `COVERING` is mechanically bullish (shorts exiting pushes price up) but tactically weak — the buying is forced, not directional. The partial-only treatment reflects this: it can help but doesn't score standalone.
- `CAPITULATION` is mechanically bearish but often marks exhaustion. Partial-only treatment again: the engine won't short capitulation outright, only with cross-confirmation.
- During cold-start warmup (`d15m = 0.000%`), `Signal: NEUTRAL` is structural — wait for the ring buffer to fill before trusting OI signals.

1.14 12. Order Flow

```
ORDER FLOW:
  Spread: 1.4 bps   |   TIGHT
 OFI Ratio: 10.08  |  Bid Vol: 583380  |  Ask Vol: 57900  |  BUY DOMINANT  |  Momentum: RISING
  CVD:        Net:182100   |   Slope:RISING   |   Div:NONE
  TFI:        0.606   |   BUY PRESSURE
  MicroCVD:   E:2840   M:15250   L:-14720   |   DECELERATING   |   BULL_DECEL
```

Five order-flow primitives, each sampling a different depth / aggressor / temporal segment.

1.14.1 Bid-Ask Spread (SpreadBps)

What: Bid-ask spread on the L2 order book snapshot, measured in basis points. Acts as an entry-side gate — wide spread fires a verdict-side penalty.

Calculation (inline in `MainForm_Analysis.RunAnalysisAsync`):

```
mid = (bestBid + bestAsk) / 2
SpreadBps = (bestAsk - bestBid) / mid × 10000
```

Computed once per run from the same `GetOrderBookAsync` snapshot used by OFI.

Classification (v17 thresholds):

Condition	Class
`SpreadBps` ≤ `cfg.Indicators.Spread.TightThresholdBps` (default 3.0)	`TIGHT`
`cfg.Indicators.Spread.TightThresholdBps` < `SpreadBps` ≤ `cfg.Indicators.Spread.WideThresholdBps` (default 5.0)	`NORMAL`
`SpreadBps` > `cfg.Indicators.Spread.WideThresholdBps`	`WIDE`

Scoring use:

- **`WIDE` penalty:** `LongScore -= cfg.Indicators.Spread.WidePenalty (default 1)` if a long verdict is dominant; mirrored for short. Penalty is dominant-side only — it does not penalise both sides nor flip directional bias.
- **`TIGHT` and `NORMAL`**: no scoring effect.

Display colour: green `TIGHT`, dim `NORMAL`, amber bold `WIDE`.

CSV column 69: `SpreadBps` — Double, 4dp.

Interpretation:

- The order book is already fetched for OFI; SpreadBps is a near-zero-cost microstructure read. The penalty exists to catch the "looks like a great breakout but the book is actually empty" trap during flush events.
- Normal BTC-PERPETUAL spread on Deribit sits 1–3 bps. Above 5 bps usually indicates an active flush / news event / deep order book withdrawal — not a clean entry environment.
- The penalty does not block a trade — at most it degrades the verdict tier by one (e.g., STRONG → MEDIUM). If the underlying score is high enough to absorb the penalty, the verdict still fires.
- During trending spread expansion (the spread widens because volatility is genuinely high but the book is not flushing), expect to see WIDE more often. The penalty is conservative — accept the false-negatives during high-vol regimes.

1.14.2 OFI (Order Flow Imbalance)

What: Book-depth-weighted bid/ask volume imbalance ratio from the L2 snapshot.

Calculation (`CalcOFI` in `Indicators_OrderFlow.vb`):

- Take the top `book_depth (5)` bid and ask levels from the order book snapshot.
- Build descending weights `{5, 4, 3, 2, 1}` — nearest level highest weight.
- `bidVol = Σ(bid.Size × weight)` over the 5 levels; `askVol = Σ(ask.Size × weight)`.
- `OFIRatio = bidVol / askVol`.
- Classification (v14 thresholds):
  - `OFIRatio > buy_dominant_ratio (2.0)` → `BUY DOMINANT`
  - `OFIRatio < sell_dominant_ratio (0.5)` → `SELL DOMINANT`
  - Else → `BALANCED`

Scoring use:

- **Full long (Microstructure)**: `BUY DOMINANT` → `+1 LongScore`.
- **Full short**: `SELL DOMINANT` → `+1 ShortScore`.
- **`BALANCED`**: no score.

Display colour: green when ratio > 1.2, red < 0.8, grey between. (Note: the visual threshold is stricter than the scoring threshold — so you can see a green ratio that hasn't yet cleared `BUY DOMINANT`. The `BUY DOMINANT` label is authoritative for scoring.)

Interpretation:

- 5-level depth means OFI is reading the *visible* book — it won't catch hidden / iceberg liquidity, but does catch the visible tilt that most liquidity-taking participants see.
- A ratio `> 5x` like the sample's `10.08` is extreme — usually the result of one side pulling liquidity (thin ask book, not necessarily heavy bid book). Cross-check the `Bid Vol` and `Ask Vol` absolute values: if the low side is very small (e.g. `57900` vs `583380`), it's more "ask pulled" than "bid stacked".
- The v14 relaxation from `3.0 / 0.333` to `2.0 / 0.5` means OFI now fires earlier. In practice this makes OFI contribute a signal in more of the "meaningful tilt" range where the old thresholds stayed silent.
- OFI is a leading indicator — imbalance visible before price moves. But on Deribit with REST polling, you're seeing a snapshot not a stream; during high volatility the snapshot can be stale by the time it arrives.

1.14.3 OFI Momentum

What: Direction of OFI signal change over the last 3 samples. RISING / FALLING / FLAT. Computed against a 10-sample ring buffer (`_ofiHistory` in `MainForm_Layout`).

Calculation (`CalcOFIMomentum` in `Indicators_OrderFlow.vb`):

- Buffer holds the last 10 numerical OFI signal codes (BUY_DOMINANT = +2, BUY_LEAN = +1, NEUTRAL = 0, SELL_LEAN = −1, SELL_DOMINANT = −2).
- Compare mean of last `cfg.Indicators.OFI.MomentumWindow` (default 3) entries against the entry `MomentumWindow + 1` positions back.
- If `delta` $\geq$ `cfg.Indicators.OFI.MomentumThreshold` (default 0.5) → `RISING`.
- If `delta` $\leq$ `-cfg.Indicators.OFI.MomentumThreshold` → `FALLING`.
- Otherwise → `FLAT`.
- If buffer has fewer than `2` $\times$ `MomentumWindow` entries → `FLAT` (warmup).

Scoring use (in Step 2 OFI block, after the level signal scores):

- **`OFI level = BULL`** AND **`Momentum = RISING`** → `LongScore += cfg.Indicators.OFI.MomentumAmplify (default 1)` capped at regimeMax. Breakdown row note: `[+momentum: amplify on rising flow]`.
- **`OFI level = BULL`** AND **`Momentum = FALLING`** → `LongScore -= cfg.Indicators.OFI.MomentumSoften (default 1)`. Note: `[-momentum: soften on fading flow]`.
- Mirror for `SELL_DOMINANT` / `SELL_LEAN`.
- **`FLAT`** → no modifier. Most common state.
- Controlled by `cfg.Indicators.OFI.MomentumEnabled`. When false, momentum is computed and displayed but not scored.

Display: Inline on the OFI line as `Momentum: RISING / FALLING / FLAT`. Colour matches OFI level direction when momentum agrees, dim otherwise.

CSV column 70: `OFIMomentum` — string enum.

Interpretation:

- OFI level measures the current imbalance; OFI momentum measures whether that imbalance is accelerating or decelerating. A level signal being amplified by rising momentum is structurally stronger than a level signal that's fading.
- The pattern matches `FundingMomentum` (same ring-buffer + windowed-delta design, see Section 15). Internal consistency was the goal — both adjuncts modify their parent score in the same way.
- A momentum shift (RISING → FALLING in consecutive runs) on a held position is an early

warning that the flow is rolling over — fires before the level signal itself flips.

- During quiet sessions where OFI level rarely leaves NEUTRAL, expect mostly `FLAT`. Don't read it as bearish or bullish.

1.14.4 CVD

What: Cumulative Volume Delta across recent trades — net signed aggressor notional, a slope classification, and a divergence state against price.

Calculation (`CalcCVD`):

- Trades fetched via `GetRecentTradesAsync(100)` (count hardcoded in `MainForm_Analysis.RunAnalysisAsync`; the v14 `cvd.trade_lookback` config key was never wired to this fetch and was removed in v15).
- Each trade: `signedDelta = +amount` if `Direction = buy` else `-amount`. (Amount is in USD notional on Deribit BTC-PERPETUAL.)
- Split window into 3 equal segments — early, mid, late thirds.
- `CVDValue = early + mid + late` (total net delta).
- Weighted slope: `weightedSlope = lateDelta × 2 − earlyDelta × 1`.
- Slope threshold: `max(slope_min_usd (12000 v14), |CVDValue| × slope_pct_of_value (0.01))`.
- `CVDSlope`:
  - `weightedSlope > +threshold` → `RISING`
  - `weightedSlope < -threshold` → `FALLING`
  - Else → `FLAT`
- `CVDDivergence`:
  - Requires `|priceChange| ≥ divergence_price_gate (0.0005)` (0.05% over the last 2 candles).
  - Price up + CVD negative → `BEARISH`
  - Price down + CVD positive → `BULLISH`
  - Else → `NONE`

Scoring use:

- **Full long (Microstructure)**: `CVDSlope = RISING` AND `CVDValue > 0` → `+1 LongScore`.
- **Full short**: `CVDSlope = FALLING` AND `CVDValue < 0` → `+1 ShortScore`.
- **Divergence penalty**: `BEARISH` → `LongScore -= DivergencePenalty (1)`. `BULLISH` → `ShortScore -= 1`.
- **Pass 2b OI × CVD**: uses `cvdBullish = (RISING AND Value > 0)` / `cvdBearish = (FALLING AND Value < 0)` as the confirmation/conflict axis against OI.
- **Pass 2c TRENDING input**: aligned if slope and sign both match dominant side.

Display colour: green on `RISING`, red on `FALLING`, grey on `FLAT`.

Interpretation:

- The 3-segment weighted slope (late × 2 − early × 1) is deliberately late-weighted. A single large trade early in the window can't dominate the classification — you need the back third of the window to be genuinely trending. This reduces false RISING / FALLING flips.
- v14's `slope_min_usd = 12000` (raised from 1000) means small order flow no longer qualifies as trending. On active BTC perp, net notional over 100 trades can easily reach 100K-500K — 12K is a meaningful floor that filters dead sessions.
- `Div: BEARISH` with `Slope: RISING` is possible — price rising + CVD rising but slower than price would expect. Penalty fires against LongScore. Read as "the rally is real but buy aggression is slowing".
- `FLAT` slope with large absolute `CVDValue` means strong directional bias built earlier in the window but current aggression is balanced — no fresh score, but the positional lean is still there.

1.14.5 TFI (Trade Flow Index)

What: Short-burst aggressor pressure ratio over the most recent 30 trades.

Calculation (`CalcTFI`):

- Take first `tfi_window_size (30)` trades from the recent-trades list.
- `buyFlow = Σ(buy amounts)`, `sellFlow = Σ(sell amounts)`.
- `TFIValue = (buyFlow - sellFlow) / (buyFlow + sellFlow)` — normalised `[-1, +1]`.
- Classification against `threshold (0.15)`:
  - `TFIValue > +0.15` → `BUY PRESSURE`
  - `TFIValue < -0.15` → `SELL PRESSURE`
  - Else → `NEUTRAL`

Scoring use:

- **Full long (Microstructure)**: `BUY PRESSURE` → `+1 LongScore`.
- **Full short**: `SELL PRESSURE` → `+1 ShortScore`.
- `NEUTRAL`: no score.

Display colour: green on `BUY PRESSURE`, red on `SELL PRESSURE`, grey on `NEUTRAL`.

Interpretation:

- TFI's 30-trade window is deliberately small — it measures very recent (often last 30–120

seconds) aggressor burst, not structural flow. Window was separated from MicroCVD (50) precisely because a burst signal and a structural segmentation signal need different horizons.

- `|TFIValue| > 0.6` (like the sample's `0.606`) is extreme — 60%+ imbalance across 30 trades means one side is clearly clubbing the other. Pair with MicroCVD direction for conviction.
- `NEUTRAL` with `|TFIValue|` just below `0.15` means there's a mild aggressive lean that didn't clear the threshold. Watch the direction — if the next run also reads mild-lean same side, the signal is building.

1.14.6 MicroCVD

What: Intra-window CVD segmentation — splits the 50-trade window into thirds and classifies whether net delta is accelerating or decelerating.

Calculation (`CalcMicroCVD`):

- Take first `micro_window_size (50)` trades.
- Split into 3 equal segments: `MicroCVDEarly` / `Mid` / `Late` — each the net signed amount for that third.
- `netDelta = Early + Mid + Late`; `isBull = netDelta > 0`.
- Momentum classification (using `accel_threshold (10000)` v14):
  - **Bull case** (`isBull = true`):
    - `Late > 0` AND `Late > Early + 10000` → `ACCELERATING`
    - `Late < 0` OR `Late < Early - 10000` → `DECELERATING`
    - Else → `FLAT`
  - **Bear case** (`isBull = false`):
    - `Late < 0` AND `Late < Early - 10000` → `ACCELERATING`
    - `Late > 0` OR `Late > Early + 10000` → `DECELERATING`
    - Else → `FLAT`
- `MicroCVDSignal` cross-tabulation:

isBull	Momentum	Signal
true	ACCELERATING	`BULL_ACCEL`
true	DECELERATING	`BULL_DECEL`
false	ACCELERATING	`BEAR_ACCEL`
false	DECELERATING	`BEAR_DECEL`
any	FLAT	`FLAT`

Scoring use:

- **Full long (Microstructure)**: `BULL_ACCEL` → `+1 LongScore`.
- **Full short**: `BEAR_ACCEL` → `+1 ShortScore`.
- **Deceleration penalty** (`DecelPenalty = 1`):
  - `BULL_DECEL` → `ShortScore -= 1`. Logic: net flow is still bullish (`isBull = true`) but weakening; shorts are clearly NOT in control, so penalise any short-side score they've accumulated. Breakdown note: `PENALTY -1 opposing`.
  - `BEAR_DECEL` → `LongScore -= 1`. Mirror.
- **FLAT stall penalty**: when `MicroCVDSignal = FLAT` AND:
  - `CurrentPrice > VWAP` AND `CVDValue` $\leq$ `0` → `LongScore -= 1`. Note: `STALL PENALTY -1 [L] (price>VWAP, CVD<=0)`. Read: price holding above VWAP but no buy flow confirms it — long posture is stalling.
  - `CurrentPrice < VWAP` AND `CVDValue` $\geq$ `0` → `ShortScore -= 1`. Mirror.
- **Hold Status microstructure exit**: `BULL_DECEL` / `BEAR_DECEL` counts as one adverse signal towards the 2-of-N fast-exit layer (see Verdict block, Hold / Exit).
- **Context tag input**: `BULL_DECEL` (if long dominant) or `BEAR_DECEL` (if short) contributes one count toward `MOMENTUM_FADING` classification. Also `MicroCVDLate < 0.5 × MicroCVDEarly` (same-side collapse) counts independently.

Display fields:

- `E` — `MicroCVDEarly` (first third's net delta, USD notional, signed)
- `M` — `MicroCVDMid`
- `L` — `MicroCVDLate`
- `DECELERATING` / `ACCELERATING` / `FLAT` — the Momentum classification
- `BULL_ACCEL` / `BULL_DECEL` / `BEAR_ACCEL` / `BEAR_DECEL` / `FLAT` — the Signal

Display colour:

- Green — `BULL_ACCEL`
- Red — `BEAR_ACCEL`
- Soft green — `BULL_DECEL`
- Soft red — `BEAR_DECEL`
- Grey — `FLAT`

Interpretation:

- **Negative segment values are valid and expected**, explicitly documented in the trader profile. `L: -14720` in the sample means the last third of the window saw net sell-side aggression — which, with a positive net delta overall (`E + M + L = 3370 > 0`), produces `isBull = true` + `L < E - threshold = 2840 - 10000 = -7160` → `DECELERATING` → `BULL_DECEL`.
- `BULL_DECEL` is visually "bullish but weakening". The scoring engine uses it to penalise *shorts* (not longs) because net bias is still up — but the context tag uses it as evidence that bullish momentum is fading. Both behaviours coexist deliberately.
- The v14 raise of `accel_threshold` from 5000 to 10000 means DECEL/ACCEL classifications now require larger segment-to-segment moves before firing — quiet sessions produce more `FLAT` signals, active sessions are unaffected. Interim-static fix until dynamic scaling ships.
- **FLAT stall penalty is subtle**: it fires when the directional structure (price vs VWAP) disagrees with accumulated flow (CVD sign). This catches the "price drifting above VWAP without buy aggression" or mirror case — the trade thesis is stalling. Rare but meaningful when it fires.
- Reading E / M / L as a trend: `E < M < L` with all positive = accelerating bull (clean). `E > M > L` with all positive = mature rally running out of energy. `E > 0, L < 0` with net positive = last-third flip (often a local top). Don't over-read — the engine's classification is the authoritative call; the numbers are for manual cross-checking.

1.15 13. Liquidations

```
LIQUIDATIONS:
  Long: 0  |  Short: 0  |  Signal: NONE
```

Penalty-only signal derived from forced-liquidation trades in the recent trade stream. No positive reward — a liquidation cascade on one side penalises that same side's score (directional conviction evaporates when forced exits dominate).

1.15.1 Long / Short / Signal

What:

- `Long` — total BTC size of long liquidations in the recent trades window.
- `Short` — total BTC size of short liquidations in the recent trades window.
- `Signal` — classification of which side's liquidations dominate, filtered by `DominanceRatio`.

Calculation (`CalcLiquidations` in `Indicators_OrderFlow.vb`):

- Iterate the recent-trades list. For each trade with `Liquidation != "none"`:

  - `Direction = "buy"` (counterparty was buying, forced short closing) → `liqShortSize += amount`.
  - `Direction = "sell"` (counterparty was selling, forced long closing) → `liqLongSize += amount`.
- Classification using `cfg.Indicators.Liquidations.DominanceRatio` (2.0 in settings.json):
  - `liqLongSize > 0` AND `liqLongSize` ≥ `liqShortSize` × `DominanceRatio` → `LONG LIQS`
  - `liqShortSize > 0` AND `liqShortSize > liqLongSize` × `DominanceRatio` → `SHORT LIQS`
  - Else → `NONE`

Scoring use:

- **`LONG LIQS`**: `LongScore -= LiqLargePenalty (2)` if `liqLongSize > LargeLiqSize (200 BTC)`, else `LongScore -= LiqStandardPenalty (1)`.
- **`SHORT LIQS`**: mirrored on `ShortScore`.
- **`NONE`**: no score change.
- Breakdown note appends `PENALTY -N [L]` or `PENALTY -N [S]` when penalty fired.

Display colour: amber when `Signal != NONE`, dim grey when `NONE`.

Interpretation:

- Penalty-only since v0.17 — was previously a two-way reward that fired ~95% of the time as non-directional padding. Now silent in the normal case; only speaks when there's an actual cascade.
- `DominanceRatio = 2.0` means the dominant side must be ≥ 2× the other before the classification fires. Mixed liquidations (say, 100 BTC long + 80 BTC short) read as `NONE`. Trader profile flagged 1.2–1.5 as a candidate tune; currently held at 2.0 pending backtesting.
- `LargeLiqSize = 200 BTC` is the threshold between `-1` (standard) and `-2` (large) penalties. At BTC $76k that's ~$15M in a single direction over the recent window — a genuine cascade event. The `-2` penalty punishes the cascade *direction* because forced exits at that magnitude imply the level just printed is a local extreme that's already been hit hard, not a fresh setup.
- An "inverted" read like `LONG LIQS` penalising LongScore can look counterintuitive — but longs being liquidated means price moved against longs, i.e. price is falling or just fell. Penalising further long entries during active long-side capitulation is correct. The mirror applies to `SHORT LIQS`.
- `Long: 0 | Short: 0 | Signal: NONE` is the normal resting state during a quiet session — not a data issue.

1.16 14. MTF Gate

```
MTF GATE (15m): PASS
  15m Trend: BULL  |  ADX: 20.5  |  EMA: BEAR
 Reason: MTF PASS [LONG] 15m +DI:25.4 -DI:20.0 ADX:20.5 EMA:BEAR | Bull:2 Bear:1 (need 2)
```

The 15m multi-timeframe confluence gate. A **hard veto** — when it blocks, the verdict is forced to `NO TRADE` regardless of 1m score.

1.16.1 Section header

Format: `MTF GATE (15m): <PASS | BLOCK>`

State:

- `PASS` — gate did not veto. Either the 15m state aligns with the proposed 1m direction, is flat (neutral), or no direction was proposed.
- `BLOCK` — gate vetoed. The 1m signal would have been `WEAK/MED/STRONG LONG` or `SHORT`, but 15m state opposed the direction.

1.16.2 15m Trend / ADX / EMA

What:

- `15m Trend` — `MTF15mTrend`, one of `BULL` / `BEAR` / `FLAT`.
- `ADX` — `MTF15mADX`, Wilder-smoothed ADX on the 15m series.
- `EMA` — `MTF15mEMAAlignment`, one of `BULL` / `BEAR` / `MIXED`.

Calculation (`CalcMTFGate` in `Indicators_Structure.vb`):

- 15m candles fetched with a 60-second TTL cache (`MTF_TTL_SECONDS`) — doesn't re-fetch on every 1m run.
- `candleLookback = cfg.MTFGate.CandleCount (60)` — last 60 × 15m candles = 15 hours of history.
- Compute `CalcDMI(window, adxPeriod=cfg.MTFGate.DmiPeriod=9)` on the 15m window → `plusDI`, `minusDI`, `mtfADX`.
- `dmiIsBull = plusDI > minusDI`.
- `adxStrong = mtfADX ≥ cfg.MTFGate.AdxMin (20.0)` — **v14 fix**: this value now reads from the dedicated `adx_min` setting. Pre-v14 it silently borrowed the 1m trend threshold (25), effectively disabling the gate's ADX component in most mid-range sessions.
- Compute `CalcEMA(window, 9/21/50)` → `emaBull = EMA9>EMA21>EMA50`, `emaBear = EMA9<EMA21<EMA50`, `MIXED` otherwise.
- **2-of-3 majority vote** accumulates `bullScore` / `bearScore`:

  - `dmiIsBull` → `bullScore += 1`, else `bearScore += 1`.
  - `adxStrong` AND `dmiIsBull` → `bullScore += 1`; `adxStrong` AND `!dmiIsBull` → `bearScore += 1`.
  - `emaBull` → `bullScore += 1`; `emaBear` → `bearScore += 1`; `MIXED` → no contribution.
- Classification against `cfg.MTFGate.RequiredConfirms (2)`:
  - `bullScore` ≥ `2` → `MTF15mTrend = BULL`
  - `bearScore` ≥ `2` → `BEAR`
  - Else → `FLAT`

Gate decision using `proposedDirection` (set upstream from 1m regime + EMA alignment):

Proposed direction	`MTF15mTrend`	Result
`LONG`	`BULL` or `FLAT`	`PASS`
`LONG`	`BEAR`	`BLOCK`
`SHORT`	`BEAR` or `FLAT`	`PASS`
`SHORT`	`BULL`	`BLOCK`
`NONE`	any	`PASS` (no direction proposed — gate inactive)

Proposed direction logic (in `MainForm_Analysis.vb`):

- `Regime = TRENDING_UP` OR 1m `EMAAlignment = BULL` → `mtfProposed = LONG`
- `Regime = TRENDING_DOWN` OR 1m `EMAAlignment = BEAR` → `mtfProposed = SHORT`
- Else → `NONE`

Veto trigger (Step 4b): The gate only actually turns the verdict into `NO TRADE` when:

- `cfg.MTFGate.Enabled = true`, AND
- A direction cleared `tWeak` on effective score, AND
- `MTFGatePass = false`.

In that case the verdict becomes `NO TRADE [WEAK LONG]` / `NO TRADE [WEAK SHORT]` (bracketed tag shows the suppressed lean).

1.16.3 Reason

What: Verbose description of the gate outcome. Always present.

Format: One of:

- `MTF PASS [LONG] 15m +DI:<+DI> -DI:<-DI> ADX:<adx> EMA:<align> | Bull:<b> Bear:<r> (need <n>)`
- `MTF BLOCK [LONG vs BEAR] 15m ...` (analogous for BLOCK)
- `MTF PASS [SHORT] ...`
- `MTF BLOCK [SHORT vs BULL] ...`
- `MTF state: <trend> | 15m ...` (when no direction proposed)
- `MTF: insufficient 15m candles (<N>)` (when fewer than `adxPeriod + 2` = 11 candles available)
- `MTF gate: no data` (cold-start placeholder before first 15m fetch)

1.16.4 Breakdown row

Rendered as `MTF Gate (15m)` in the signal breakdown table. `[L]` / `[S]` hit:

- `LongHit = MTFGatePass AND proposedDirection = LONG`
- `ShortHit = MTFGatePass AND proposedDirection = SHORT`

Note field is the same `MTFGateReason` string as rendered in the MTF section.

Display colour: green on `PASS`, red on `BLOCK` for the 15m Trend / ADX / EMA line. Reason line always rendered dim grey (it's explanatory text, not a primary value).

1.16.5 Interpretation

- The gate is *soft* by design: `FLAT` on 15m always passes, because a flat higher timeframe neither confirms nor disconfirms the 1m setup. The veto only fires on active disagreement (15m `BULL` vs proposed `SHORT`, or vice versa).
- The sample output shows a subtle case: 15m Trend says `BULL`, but EMA says `BEAR`. Bull:2 came from `dmiIsBull` + `adxStrong confirm` (two votes); bear:1 came from `emaBear`. The 2-of-3 majority still tilts bull, gate passes LONG. Read as "DMI/ADX say bull, but the EMA structure hasn't caught up yet" — a regime in early transition.
- A `BLOCK` event is the one place where a strong 1m score gets silenced. If you see a `NO TRADE [WEAK LONG]` with MTF `BLOCK`, the trade-off is clear: the 1m pipeline liked it, the 15m pipeline hated it. Trust the higher timeframe.
- The 60-second TTL cache means the 15m values can be up to 60 seconds stale relative to the 1m fetch. On the 15m timeframe this is imperceptible (candles are 15 min each). The cache exists to save API quota — 15m candles don't move enough to warrant fetching every 1m run.
- If `Reason` reads `MTF: insufficient 15m candles (N)`, the gate opens (PASS) and does not veto — it's a degraded-mode fallback. Watch for this on cold start or after a long disconnect.

1.17 15. Funding

```
FUNDING:
  Rate: 0.0007%  |  NEUTRAL
  Momentum: FLAT  |  Enabled: YES  |  Soften: +1  |  Amplify: -1
```

Two-row funding block: the raw rate + crowd bias on the first row, and momentum metadata (Step 3b inputs) on the second.

1.17.1 Row 1 — Rate / Bias

What:

- `Rate` — the funding rate, displayed as percent (`FundingRate × 100`, formatted to 4 decimal places).
- `Bias` — `r.FundingBias`, one of `NEUTRAL` / `LONGS CROWDED` / `LONGS HEAVILY CROWDED` / `SHORTS CROWDED` / `SHORTS HEAVILY CROWDED`.

Calculation:

- `FundingRate` fetched from Deribit via `GetFundingRateAsync` — raw 8h-scale funding value. Published roughly every 8 hours by Deribit; samples between publications return the same value.
- `FundingBias` classified elsewhere in `MainForm_Analysis` by comparing `FundingRate` to crowd thresholds. `HEAVILY CROWDED` requires a larger magnitude than plain `CROWDED`.

Scoring use (Step 3 — baseline funding modifier, in `ScoringEngine_Calculate_Scoring.vb`):

Condition (v14 thresholds)	Action
`Rate > funding_high_positive (0.0003)`	`ls -= FundingHighPenalty (2)`, `ss += FundingHighBoost (1)`. Note: `STEP3: -2[L] +1[S]`
`Rate > funding_low_positive (0.00005)`	`ls -= FundingLowPenalty (1)`. Note: `STEP3: -1[L]`
`Rate < funding_high_negative (-0.0003)`	`ss -= FundingHighPenalty (2)`, `ls += FundingHighBoost (1)`. Note: `STEP3: -2[S] +1[L]`
`Rate < funding_low_negative (-0.00005)`	`ss -= FundingLowPenalty (1)`. Note: `STEP3: -1[S]`
Else	No change. Note: `STEP3: none`

Score is post-clamped to `max(0, ...)`.

Display colour (Row 1):

- Red — any `HEAVILY CROWDED` bias.
- Grey — `NEUTRAL`.
- Amber — plain `CROWDED` on either side.

Interpretation:

- **v14 raised the high-threshold from 0.0001 to 0.0003** — BTC perp routinely sits at ±0.01%/8h (= 0.0001 raw), which previously fired the max penalty every run. Post-v14, only genuinely extreme funding ($\geq$ 0.03%/8h = 0.0003 raw) fires the `FundingHighPenalty (2)` + `FundingHighBoost (1)` contrarian tilt.
- Contrarian framing: extreme positive funding → longs overcrowded → small nudge *against* longs, *for* shorts. The boost is deliberately smaller than the penalty (1 vs 2) to reflect asymmetric conviction.
- Funding is adjunct — it never votes in Step 2 scoring (rejected pattern; would double-count). Row 1 breakdown appears as `Funding (info)` with no `[L]` / `[S]` hit marker, carrying the STEP3 + STEP3b notes as informational text only.
- The rate displayed `0.0007%` = raw `0.000007`, well below any threshold — `STEP3: none` is correct.

1.17.2 Row 2 — Momentum / Enabled / Soften / Amplify

What:

- `Momentum` — `r.FundingMomentum`, one of `RISING` / `FALLING` / `FLAT`.
- `Enabled` — `YES` / `NO`, from `cfg.Indicators.Funding.MomentumEnabled`.
- `Soften` — displayed as `+N`, from `cfg.Indicators.Funding.MomentumSoften` (default 1).
- `Amplify` — displayed as `-N`, from `cfg.Indicators.Funding.MomentumAmplify` (default 1).

Calculation (`CalcFundingMomentum` in `Indicators_OrderFlow.vb`):

- Uses `_fundingHistory` ring buffer (max 10 samples).
- v14 dedup: only appends `fundingRate` to history when the value has actually changed from the previous sample (Deribit publishes every ~8h, not every 1m).
- Requires $\geq$ 2 distinct samples; cold start returns `FLAT`.
- `window = cfg.Indicators.Funding.MomentumWindow (3)`, `threshold = MomentumThreshold (0.0001)`.
- `delta = history[last] - history[max(0, count - 1 - window)]`.
- Classification:
  - `delta > +0.0001` → `RISING`
  - `delta < -0.0001` → `FALLING`
  - Else → `FLAT`

Scoring use (Step 3b — funding momentum modifier):

Applied on top of Step 3. Behaviour depends on `FundingBias` + `FundingMomentum`:

Bias	Momentum	Action
`LONGS CROWDED` / `LONGS HEAVILY CROWDED`	`RISING`	`ls -= min(MomentumAmplify, FundingHighPenalty)` (amplifies crowding penalty; capped at high penalty to avoid double-counting). Note: `STEP3b: -1[L] crowding↑`
`LONGS ...`	`FALLING`	`ls += MomentumSoften (1)` (de-crowding; capped at `regimeMax`). Note: `STEP3b: +1[L] de-crowding`
`SHORTS CROWDED` / `SHORTS HEAVILY CROWDED`	`FALLING`	`ss -= min(MomentumAmplify, FundingHighPenalty)`. Note: `STEP3b: -1[S] crowding↓`
`SHORTS ...`	`RISING`	`ss += MomentumSoften`. Note: `STEP3b: +1[S] de-crowding`
`NEUTRAL`	`RISING` AND `Rate > 0`	`ls -= amplify` (neutral→crowding transition). Note: `STEP3b: -1[L] neutral→crowding`
`NEUTRAL`	`FALLING` AND `Rate < 0`	`ss -= amplify`. Note: `STEP3b: -1[S] neutral→crowding`
Any	Other combinations	No change. Note: `STEP3b: none`

Enabled gate: entire Step 3b skipped if `cfg.Indicators.Funding.MomentumEnabled = false`. Note: `STEP3b: disabled`.

Display colour (Row 2):

- Amber — `RISING` (penalty-amplifying direction).
- Green — `FALLING` (de-crowding direction).
- Grey — `FLAT`.

Display convention for Soften / Amplify:

- Displayed as `Soften: +N` and `Amplify: -N` even though both are stored as positive magnitudes in config. The sign convention reflects their **effect on score** — soften *adds back*, amplify *deducts further*. This is a readability choice, not a sign error.

Interpretation:

- **Dedup matters**: pre-v14, `_fundingHistory` filled with 10 identical 1m snapshots of the same 8h-published rate — `delta` was always 0 → `FLAT` → Step 3b almost never fired. Post-v14, history only grows on genuine rate transitions, so `RISING`/`FALLING` classifications actually correspond to Deribit publishing a changed funding rate. Expect Step 3b to be dark between funding publications and active only at the boundaries.
- Asymmetric amplify/soften logic: `Amplify` is capped at `FundingHighPenalty (2)` to prevent a compounding penalty (baseline + momentum) from obliterating the score on a mild crowding event. `Soften` has no explicit cap but is subject to the `regimeMax` ceiling.
- The NEUTRAL + RISING + Rate>0 case catches the *transition into* crowding — funding is still in the neutral band but momentum is building toward positive. Engine pre-penalises the long side before the rate actually crosses the high threshold. This is the one place Step 3b fires in a neutral session.
- `Enabled: NO` effectively disables this whole row's scoring contribution. Use to isolate Step 3b's effect when debugging or during periods of suspect funding data.

1.18 16. Signal Breakdown Table

```
============================================================
  SIGNAL BREAKDOWN
============================================================
  Signal               Long   Short  Note
  ------------------------------------------------------------------------
  ROC(9)                [L]          0.115 | Slope: FLAT | PARTIAL->UPGRADED [L]
  RSI(9)                             52.6 | zones OB:60 OS:40 | DIV:BEARISH
  ...
  ------------------------------------------------------------------------
  TOTAL                   8       2
```

The itemised scoring ledger. Every signal that contributes to (or penalises) the score appears as a row; the `TOTAL` row at the bottom must match the header's `LongScore` / `ShortScore`.

1.18.1 Layout

Columns:

- `Signal` — 18-char left-justified label.
- `Long` — 5-char field, shows `[L]` if `item.LongHit = true`, else blank.
- `Short` — 6-char field, shows `[S]` if `item.ShortHit = true`, else blank.
- `Note` — free text; per-indicator diagnostic string.

Row colour:

- `C_HIT` (brighter) — row has a hit (`LongHit` or `ShortHit`).
- `C_DIM` — no hit (score-neutral or penalty-only row).

1.18.2 Rows (in display order)

Populated by `RunScoringPipeline` in `ScoringEngine_Calculate_Scoring.vb` (rows 1–20) and `Calculate` in `ScoringEngine_Calculate_Verdict.vb` (MTF Gate). Row 21 (Regime Align) appears only conditionally.

#	Label	Hit logic (source)	Note content
1	`ROC(9)`	Full `rocLong/rocShort` OR upgraded partial via Pass 2	`<ROC:F3> | Slope: <ROCSlope>` + optional partial/upgrade tag
2	`RSI(9)`	Full `rsiLong/rsiShort` OR upgraded partial	`<RSI:F1> | zones OB:60 OS:40` + divergence tag + penalty tag + partial/upgrade tag
3	`DMI +/-DI`	`dmiLong/dmiShort`	`+DI:<+DI> -DI:<-DI>`
4	`ADX><threshold>`	`adxLong/adxShort` (requires `ADX > adxTrend` AND dmi aligned)	`<ADX:F1> | thr:<adxTrend:F0>`
5	`Volume`	Full `volLong/volShort` OR upgraded mid-tier	`<VolumeRatio:F2>x | thr H:<volHigh>x M:<volMid>x [<normMode>]` + partial/upgrade tag

#	Label	Hit logic (source)	Note content
6	VWAP	Full vwapLong/vwapShort OR upgraded partial	During warmup: WARMUP (N/15 candles) -- signal suppressed; otherwise price/VWAP/band summary + partial/upgrade tag
7	BBW/TTM	bbwLongHit/bbwShortHit (fires only on RELEASING/NONE + BUILDING)	BBW value + SqueezeStatus + TTM summary + penalty-active indicator
8	EMA 9/21/50	emaBull/emaBear	9:<e9> 21:<e21> 50:<e50> | <alignment>
9	Funding (info)	**Never hits** (display-only row)	Rate%, Bias, Momentum, STEP3: / STEP3b: notes
10	OI Delta	Full oiLong/oiShort OR upgraded partial	15m:<d15>% 60m:<d60>% | <OISignal> + partial/upgrade tag + Pass 2b confirm/conflict tag
11	OFI	ofiBuy/ofiSell	Ratio:<ratio> | <OFISignal>
12	CVD	cvdLong/cvdShort	Net:<cvd> | Slope:<slope> | Div: + divergence penalty tag
13	TFI	tfiLong/tfiShort	<TFIValue:F3> | <TFISignal>
14	MicroCVD	microLong/microShort (ACCEL only)	E:<e> M:<m> L:<l> | <Momentum> | <Signal> + decel penalty tag + stall penalty tag

#	Label	Hit logic (source)	Note content
15	Liq Penalty	liqLongPenalty>0 / liqShortPenalty>0 (penalty side marked as hit)	L:<long> S:<short> | <LiqSignal> + per-side penalty tag
16	5m EMA(200)	ema200Bull/ema200Bear	<EMA200_5m> | <PriceVsEMA200>
17	Donchian(20)	Full donchLong/donchShort OR upgraded partial	U:<upper> L:<lower> | <DonchianSignal>; MID-CHANNEL -- no signal when DonchianSignal = NONE
18	OBV	obvLong/obvShort OR upgraded partial	Trend:<trend> Div: + [upgrade blocked] tag when adverse divergence + partial/upgrade tag
19	VPFR-lite	vpfrLong/vpfrShort	POC:<poc> | <VPFRSignal> | HVN@POC:<YES/NO>
20	MTF Gate (15m)	MTFGatePass AND proposedDir=LONG/SHORT	MTFGateReason (pass/block text with DMI/ADX/EMA breakdown)
21	Regime Align (2c)	regimeAlignLongHit/regimeAlignShortHit — **row appears only when Pass 2c fired** (aligned or conflicted)	REGIME ALIGN [<regime>: <sigs> ✓<suffix>] or -N REGIME CONFLICT [...: <sigs> ×<suffix>]

1.18.3 Note annotations — conditional tags

The Note field carries cross-cutting annotations that only appear in specific circumstances.

Tag	Meaning	When it appears
`PARTIAL [L*] / [S*]`	Partial signal detected but not upgraded this run	ROC, RSI, VWAP, OI, Donchian, Volume (mid), OBV
`PARTIAL->UPGRADED [L] / [S]`	Partial was upgraded to full via Pass 2 cross-category confirmation	Same set as above
`[upgrade blocked]`	OBV has adverse divergence, blocking cross-category upgrade	OBV only
`WARMUP (N/15 candles) -- signal suppressed`	VWAP session candle count below warmup threshold	VWAP only
`MID-CHANNEL -- no signal`	Price in middle half of Donchian channel	Donchian only
`DIV:<state>`	RSI divergence detected	RSI only (when `RSIDivergence != NONE`)
`PENALTY -N [L] / [S]`	Direct score penalty applied	RSI (RSI-div-plus-extreme), CVD (divergence), MicroCVD (DECEL opposing), Liq (size-dependent)
`STALL PENALTY -N [L] / [S]`	MicroCVD `FLAT` with price/CVD direction mismatch	MicroCVD only
`PASS2b: +N[L] / [S] OI×CVD confirmed`	OI full-or-upgraded + CVD aligned → `UpgradeBonus`	OI Delta row only
`PASS2b: -N[L] / [S] OI×CVD conflict`	OI full (not partial) + CVD opposed → `ConflictPenalty`	OI Delta row only
`+N REGIME ALIGN [...]`	All Pass 2c signals aligned with dominant side	Regime Align (2c) row — only rendered when gate fired
`-N REGIME CONFLICT [...]`	All Pass 2c signals conflict with dominant side	Regime Align (2c) row
`<regime>: EMA+ROC+CVD / EMA+CVD`	Pass 2c TRENDING signal set; `EMA+CVD` when ROC was inactive	Regime Align row only
`<regime>: VWAP+RSI+DON / RSI+DON`	Pass 2c RANGE_BOUND signal set; `RSI+DON` when VWAP was in warmup	Regime Align row only
`(ROC neutral) / (VWAP warmup)`	Suffix clarifying why the “reduced” signal set was used	Regime Align row only
`[LIVE] / [STATIC]`	DynamicNorms mode	Volume row only

1.18.4 TOTAL row

Format: `TOTAL <LongScore> <ShortScore>` — displayed in white bold.

Authority: These are `v.LongScore` / `v.ShortScore` — the **raw** scores after all Step 2 scoring, Pass 2/2b/2c adjustments, Steps 3/3b funding modifiers. They are the same values shown in the header `SCORE` line's non-effective fields.

Reconciliation check: Sum the `[L]` marks in the table (each = `+1`) and subtract any `PENALTY` / `CONFLICT` amounts that landed on the long side. The result should equal `LongScore`. Same for short. Small discrepancies usually mean a penalty applied to a score that was already `0` and got floored (e.g. the sample's MicroCVD `BULL_DECEL -1 [S]` which landed when short was at 0, so the penalty was absorbed — the `[L]` marks sum correctly, and short=2 is explained by 5m EMA(200) + VPFR with the micro penalty having been zero-floored at the moment it fired).

1.18.5 Interpretation

- Read the table top-down: the top rows are the core trend/momentum primitives, the middle rows are order flow, the bottom rows are structural context + gates. A verdict leaning one way with hits clustered in a single section (e.g. all order flow, no structural) is the classic `FLOW_UNCONFIRMED` setup — the CONTEXT tag will usually call it out, but the breakdown tells you *where* the unconfirmed side is thin.
- The `Funding (info)` row is the trader's readout of what Step 3 / Step 3b just did to the score — it's the only way to see funding modifier magnitudes directly. No `[L]` / `[S]` marker, but the note text is informative.
- Any line carrying `PENALTY`, `STALL`, `CONFLICT`, or `PASS2b: -N` is a score deduction — you won't find these in the TOTAL reconciliation as positive hits, but they're part of why TOTAL may be lower than the raw `[L]` count.
- The `Regime Align (2c)` row is a binary event indicator — **its presence alone is notable**. Most runs don't fire it (needs all active Pass 2c signals to unanimously agree or unanimously conflict). When you see it, the engine is expressing strong regime conviction.
- When debugging a surprising verdict, the fastest path is: (1) scan the TOTAL row to see the raw score, (2) check the `SCORE` header line for `(eff.N)` delta, (3) look for any `PASS2b`, `REGIME CONFLICT`, `PENALTY` tags in the note column to explain the delta, (4) if everything reconciles and the verdict still surprises you, check the `CONTEXT` line — it may be flagging a quality issue the raw score doesn't capture.

1.19 17. CSV Logging

The engine appends one row per analysis run to `bin/Debug/net8.0-windows/analysis_log.csv`. Schema is **v0.4 with d1 extension** — 87 columns. Used by the offline analysis script (Section 18) and the auto-tweaker (Section 19).

1.19.1 Schema versioning and rotation

`AnalysisLogger.EnsureLogFile()` reads the first line of the existing log on startup. If the header doesn't match the current expected v0.4 header, the existing file is renamed to `analysis_log.csv.<schema-tag>.bak` and a fresh file is started with the current header. Idempotent — if no log exists, just writes the header.

Two backups commonly present after Bundle 1 + Bundle 3 shipped:

- `analysis_log.csv.v0.3.bak` — pre-Bundle-1 log with 68 columns
- `analysis_log.csv.v0.4.bak` — post-Bundle-1 log with 86 columns (rotated when d1 added column 87)

1.19.2 Schema (87 columns)

```
1    Timestamp                  ISO 8601 UTC
2    Price                      Last transacted price at run time
3    Verdict                  String: STRONG_LONG / LONG / WEAK_LONG / NO_TRADE / etc.
4    Confidence                 String: HIGH / MEDIUM / LOW / N/A
5    LongScore                  Raw long score after Pass 2/2b/2c + funding modifiers
6    ShortScore                 Raw short score (same point in pipeline)
7    EffectiveLongScore         After Step 4 regime veto + Step 4b MTF veto
8    EffectiveShortScore        Same
9    MaxScore                   Regime-adjusted ceiling
10   RegimePenalty              TRANSITIONAL ADX-proximity penalty (0 elsewhere)
11   Regime                   TRENDING_UP / TRENDING_DOWN / RANGE_BOUND / TRANSITIONAL
12-13  ADX, PlusDI, MinusDI    DMI on 5m
15-16  ROC, ROCSlope           1m rate-of-change
17-18  RSI, RSIDivergence      1m RSI(9)
19   VolumeRatio                vs Volume SMA(9)
20-26  VWAP fields              Value, dev%, candle count, s1/s2 bands
27-31  BBW + TTM fields         Squeeze status, histogram, direction, signal
32-37  EMA fields               EMA9/21/50, alignment, 5m EMA200, price-vs-200
38-39  Funding rate + bias
40-43  OI fields                Current, 15m delta, 60m delta, signal
```

```
44-47  OFI fields              Ratio, bid vol, ask vol, signal
48-50  CVD fields              Value, slope, divergence
51-53  Liquidation fields      Long size, short size, signal
54-56  Donchian fields         Upper, lower, signal
57-58  OBV fields              Trend, divergence
59-63  MTF Gate fields         Pass, 15m trend, 15m ADX, 15m EMA alignment, reason
64-65  ATR + multiplier
66 VerdictContext         FLOW_UNCONFIRMED / MOMENTUM_FADING / STRUCTURALLY_WEAK / CONFIRMED (v0.3)
67  FundingMomentum            RISING / FALLING / FLAT (v0.3)
68  OiCvdOutcome               Pass 2b output (v0.3)
69  SpreadBps                  Bid-ask spread in basis points (v0.4)
70  OFIMomentum                RISING / FALLING / FLAT (v0.4)
71  FundingDelta               Period-over-period funding rate change (v0.4)
72-73  VPFRVAH, VPFRVAL        Value area boundaries (v0.4)
74-75  VPFRNearestHvnAbove/Below   Nearest high-volume walls (v0.4)
76-79  LastSwingHigh/Low 5m + 15m   (v0.4)
80-83  SwingTargetLong/Short, SwingStopLong/Short    (v0.4)
84  TargetCapReason           swing / hvn / poc / none --- Step 5b 3-tier winner (v0.4)
85-86  BestPivotByVolume5m, BestPivotVolumeRatio5m   (v0.4 reserved, populated by D2)
87  TrendStructure5m       UPTREND / DOWNTREND / EXPANSION / CONTRACTION / UNDEFINED (Bundle 3 d1)
```

Skipped runs do not write a row. When `RunAnalysisAsync` aborts due to a missing required result (1m / 5m candles, funding, book summary, order book, recent trades), the CSV is untouched.

Companion log. `settings_snapshots/manifest.csv` is a separate CSV maintained by the auto-tweaker (see §20). It is not part of `analysis_log.csv` — different schema, different cadence — but it lives in the repo alongside the snapshot `.json` files it indexes.

1.19.3 CalibrationReport

Generated on demand from the status bar `Calibration Check` link. Counts coverage along multiple axes against gate thresholds:

- **Total rows** $\geq$ 300 (gate)
- **Sessions (UTC days)** $\geq$ 3 (gate)
- **Liquidation events** $\geq$ 2 (gate; rare-event blocker — accepted as deferred 2026-05-05)
- **Per-regime row counts** $\geq$ 50 each (gate)
- Plus distribution sections (informational only): regime, verdict context, funding momentum, OI$\times$CVD outcomes, spread, OFI momentum, target cap reason, trend structure, best volume

pivot.

Verdict line at the end reads `READY` or `NOT YET READY`.

1.20 18. Analysis Report Viewer

The status bar `Analysis Report` link runs the offline analyser (`analysis/AnalysisRunner.Run`) over the current `analysis_log.csv` and opens a non-modal viewer (`AnalysisReportForm`) with the rendered markdown.

1.20.1 Output files

Written to the engine's working directory each click:

- `analysis_report_<yyyyMMdd_HHmmss>.md` — full markdown report
- `analysis_summary_<yyyyMMdd_HHmmss>.csv` — flat failure-rate matrix consumed by the auto-tweaker

1.20.2 Report sections

1. **Summary** — rows in CSV, rows excluded (forward window incomplete or session boundary), verdict tier counts, headline failure rates.
2. **Failure-Rate Matrix** — per verdict tier × hold window (5/10/15 min) × ATR threshold (0.3, 0.5, 0.8). Each cell shows `rate% (n=sample) [ci_low - ci_high]` using a 95% Wilson score interval. Highlighted cells = lowest CI width with `n ≥ 30`.
3. **Recommended (window, threshold) per tier** — auto-pick logic (lowest CI width subject to `n ≥ 30`). Used by the auto-tweaker.
4. **Verdict Context Tag × Outcome** — failure rate per VerdictContext at the recommended cell.
5. **Funding Momentum Diagnostic** — empirical FundingDelta distribution, percentile table, recommended threshold or "no change — ceiling is polling cadence" verdict.
6. **OFI Outlier Audit** — count of `OFIRatio > 100` and `> 1000`. Top 10 outliers with timestamps and raw bid/ask volumes.
7. **OI×CVD Asymmetry Audit** — confirmed_long vs confirmed_short counts, broken down by Regime and Funding Bias. Verdict: regime-period bias / asymmetric algorithm / inconclusive.
8. **Hold Window Selection Stats** — per verdict tier, percentage of runs where the recommended window was 5m / 10m / 15m. Direct trader use: "STRONG LONG verdicts are most reliable held for 10m" comes from this section.
9. **Pending data** — tier × window cells where `n < 30` (insufficient sample). User waits for more rows before treating recommendation as stable.

1.20.3 Failure definition (constants in `analysis/AnalysisConstants.vb`)

```
Public Const StrongAtrThresholds As Double() = {0.3, 0.5}
Public Const MediumAtrThresholds As Double() = {0.5, 0.8}
Public Const HoldWindowsMinutes As Integer() = {5, 10, 15}
Public Const MinSamplesPerCell As Integer = 30
Public Const MinSamplesForAutoTweakerTrigger As Integer = 60
```

For a row with verdict tier T and forward return `fr_W` at window W:

- LONG failure: `fr_W × Price < -threshold × ATR`
- SHORT failure: `fr_W × Price > +threshold × ATR`
- STRONG verdicts use the tighter `{0.3, 0.5}` set; MEDIUM uses the looser `{0.5, 0.8}`.
- `NO_TRADE` and `WEAK_*` rows are excluded from the failure-rate denominator. Tracked in informational counters only.

1.20.4 Picked-cell history

`analysis/picked_cell_history.csv` (gitignored): one line per auto-tweaker run with timestamp, tier, window, threshold. Drives the "Hold Window Selection Stats" report section.

1.20.5 Portability

All classes in `analysis/` are host-agnostic except `AnalysisReportForm.vb` (the viewer). The non-form classes are referenced from both the WinForms app and the auto-tweaker console app. Future Linux CLI port reuses them directly.

1.21 19. Tweak Settings & Auto-Tweaker

The auto-tweaker is a separate .NET 8 console app (`tools/AutoTweaker/AutoTweaker.dll`, target framework `net8.0` — no Windows dependency) that periodically reviews verdict accuracy and proposes targeted `settings.json` adjustments via the Anthropic API. Configured and triggered via the `Tweak Settings` dialog in the main window.

1.21.1 Architecture

```
tools/AutoTweaker/
├── AutoTweaker.vbproj          .NET 8 console project, zero WinForms refs
├── AutoTweakerProgram.vb      Entry point. Walks up to find DeribitVerdictEngine.sln
│                          (sets working dir), loads tweaker_config.json, invokes Core.
├── AutoTweakerCore.vb         Pipeline. Eligibility → window → failure rate → trigger
│                           → prompt → API call (or dry-run write) → diff parse → apply.
```

```
├── PromptBuilder.vb          Builds system + user message from settings.json + recent
│                             CSV slice + failure-rate matrix + picked-cell history +
│                            trader-profile constraints (rejected approaches inlined).
├── ClaudeApiClient.vb        /v1/models discovery for latest Opus, /v1/messages call.
│                               API key from ANTHROPIC_API_KEY env var only.
├── SettingsDiffApplier.vb     Validate + apply diff. Hard rejection list, 3-key cap,
│                               stale-value check, version-monotonicity bump.
├── TweakerConfig.vb          POCO for tweaker_config.json. Hot-read each invocation.
└── TweakerState.vb            POCO for state.json. Persistent across runs.
```

1.21.2 Eligibility checks (run at start of every invocation)

1. **Cooldown:** `(current CSV row count) - (state.last_run_csv_row_count)` ≥ `tweaker_config.cooldown_rows` (default 10).
2. **Session-aligned window:** walk back from end of CSV. The last `window_size_verdicts` rows (default 120) must all fall within the same UTC session bucket per `cfg.SessionVolume.Sessions[]`. If a session boundary appears earlier, the run is ineligible until the current session accumulates a full window.
3. **Tier-eligible row count:** within the window, count STRONG_* + MEDIUM_* verdicts. Must be ≥ `min_tier_eligible_rows` (default 60). Prevents auto-tweaks driven by edge cases where most rows are NO_TRADE.

If any check fails, exit code 2 (INELIGIBLE), no API call, no settings change.

1.21.3 Trigger logic

After eligibility, compute the failure-rate matrix over the window (reuses `analysis/FailureRateMatrix.vb`). Pick the most stable cell per tier (lowest CI width with `n ≥ 30`). Aggregate failure rate is the weighted mean across picked cells.

If aggregate < `failure_rate_threshold_pct` (default 40%) → exit code 0, outcome `BELOW_THRESHOLD`. Engine is performing fine, no tweak needed. This is the normal happy state.

If above → proceed to prompt build + API call.

1.21.4 Latest-Opus model resolution

`ClaudeApiClient.ResolveLatestOpusModel()`:

1. `GET /v1/models` with `x-api-key: $ANTHROPIC_API_KEY`.
2. Filter `data[]` for `id` starting with `claude-opus-`.
3. Sort by `created_at` descending.

4. Return `data[0].id`. Cache for the process duration; refetch on next process start.
5. Fallback sentinel `claude-opus-latest` if API call fails.

Version-agnostic by design — a future `claude-opus-5-0-20271015` is picked up automatically on next run.

1.21.5 Dry-run mode

When `tweaker_config.dry_run_enabled = true` (default for fresh installs):

- Full prompt + JSON request body written to `tools/AutoTweaker/dry_run_payloads/<yyyyMMdd_HHmmss>.txt`
- File contains: trigger reason, system message, user message, JSON body, instructions for human
- No API call made
- State updated: `last_run_outcome = DRY_RUN_WRITTEN`

User opens a separate Claude conversation, pastes the messages, gets back a JSON diff, saves it at `tools/AutoTweaker/manual_diffs/<timestamp>.json`, then runs `AutoTweaker.exe --apply-manual <path>` to apply.

1.21.6 SettingsDiffApplier — hard rejection list

Validates the diff before any application. Returns invalid (exit code 1) if any of:

- **3-key scope cap** — proposal touches more than 3 keys
- **Banned path fragments** — any of: `_fixed_pct_`, `bbw_none_bonus`, `oi_prev15m`, `oi_prev60m`, `atr_avg20d`, `static_vol_high`, `static_vol_mid`, `static_vol_low` (last 6 are dead v15 cleanup keys; first two are explicitly rejected patterns)
- **Disabling gated paths** — `mtf_gate.enabled = false` or `regime_weights.enabled = false`
- **Direct version edit** — applier manages version bump; diff must not touch `version`
- **Stale diff** — proposed `old_value` doesn't match current settings value

1.21.7 Apply path

When `auto_commit_enabled = true` and the diff passes validation:

- Bump `settings.json.version` by 1
- Set `modified_by = "auto-tweaker-vN"` where N = new version
- Append `change_log` entry: timestamp + summary + cited failure rate + Claude reasoning excerpt
- Write file. `SettingsLoader` FileSystemWatcher hot-reloads the engine on its next run.

When `auto_commit_enabled = false`:

- Diff written to `tools/AutoTweaker/proposed_diffs/<timestamp>.json`
- State updated with `last_pending_diff_path`
- User reviews, applies via `AutoTweaker.exe --apply-manual <path>` if accepted.

1.21.8 TweakSettingsForm controls

Non-modal dialog opened from the `Tweak Settings` link.

Control	Binds to (in `tweaker_config.json`)
`chkAutoCommit` (Checkbox)	`auto_commit_enabled` (default false)
`chkDryRun` (Checkbox)	`dry_run_enabled` (default true)
`txtWindowSize` (TextBox, int $\geq$ 10)	`window_size_verdicts` (default 120)
`txtFailThreshold` (TextBox, int 1–99)	`failure_rate_threshold_pct` (default 40)
`txtCooldownRows` (TextBox, int $\geq$ 1)	`cooldown_rows` (default 10)
`txtSnapshotStreakX` (TextBox, int $\geq$ 1)	`snapshot_streak_x` (default 3)
`txtMaxKeysPerProposal` (TextBox, int $\geq$ 1)	`max_keys_per_proposal` (default 3 — previously hard-coded)
`txtStreakWeight` (TextBox, double $\geq$ 0)	`streak_weight` (default 1.5)
`lblActiveSnapshot` (read-only)	Live: `Streak: N/X  Active snapshot: <filename | none>`
`btnShowRoundStats` (Button)	Opens `RoundStatsForm` non-modally with the last 5 rounds
`btnOpenSnapshotsDir` (Button)	Opens `settings_snapshots/` via `Process.Start`
`lblConfigPath` (read-only)	full path to `tweaker_config.json`
`lblCsvPath` (read-only)	full path to `analysis_log.csv`
`lblStatePath` (read-only)	full path to `state.json`
`lblTweakerStatus` (dynamic)	`Ready` / `Cooldown: N rows remaining` / `Waiting for session-aligned window: M/120 rows` / `Insufficient tier-eligible rows: K/60`
`btnRunNow` (Button)	Disabled unless status is `Ready`. On click: `Process.Start(AutoTweaker.exe)`

Control	Binds to (in `tweaker_config.json`)
`btnSave` (Button)	Validates input, writes to `tweaker_config.json`, MessageBox confirmation
`lblLastTweakSummary` (multi-line read-only)	`last_run_at_iso` + `last_run_outcome` + `last_proposal_summary`

The four new fields (`snapshot_streak_x`, `streak_weight`, `streak_length_clamp`, `max_keys_per_proposal`) live alongside the existing `tweaker_config.json` settings. `max_keys_per_proposal` is the previously-hard-coded `3` lifted to configurable; the conservative-bias safeguard is preserved as a default of 3.

1.21.9 Status polling

`lblTweakerStatus` updates on:

1. Subscribe to `MainForm.AnalysisCompleted` event (raised by `RunAnalysisAsync` at the end of every analysis whether successful or skipped). Update on event.
2. 30-second `System.Windows.Forms.Timer` fallback when the dialog is open.

`UpdateStatusLabel()` reads `state.json` and inspects current CSV row count + session alignment + tier-eligible counts. Cheap I/O.

1.21.10 Outcomes (state.json `last_run_outcome` field)

- `BELOW_THRESHOLD` — engine performing fine, no tweak needed (happy state)
- `INELIGIBLE` — cooldown / session-not-aligned / insufficient tiers
- `DRY_RUN_WRITTEN` — payload file generated, awaiting manual handling
- `PROPOSED` — diff parked, awaiting manual apply (auto_commit off)
- `APPLIED` — settings updated, version bumped, change_log appended
- `ERROR` — API call or validation failure

1.21.11 Linux portability

`tools/AutoTweaker/AutoTweaker.vbproj` targets `net8.0` (no `-windows` suffix). Zero WinForms references — confirmed by build inspection. Runs unmodified under `dotnet AutoTweaker.dll` on Linux. Future port: same codebase, scheduled via cron or systemd timer instead of WinForm Process.Start.

1.21.12 Constraints from trader-profile

The auto-tweaker is bounded by the same conservative-bias rules as manual tuning:

- No false-positive growth — auto-tweaker should optimise toward maintaining or *raising* the false-positive bar
- No double-counting reintroduction — the rejected-pattern list catches the obvious cases; reviewer (human if `auto_commit = false`) catches subtler ones
- Per-proposal key cap (default 3, configurable via `max_keys_per_proposal`) is the conservative-bias safeguard at the structural level — small steps, frequent review. Reverts bypass the cap because the snapshot's provenance is the validation gate (§20g).
- Latest-Opus auto-discovery means the tuning quality scales with model improvements over time without code change

---

1.22 20. Settings Snapshot History

When the auto-tweaker has run for several consecutive `BELOW_THRESHOLD` rounds without proposing a change, the engine treats those settings as "proven" under the current market regime and saves a snapshot. Snapshots are bucketed by `regime × volatility tier` (12 buckets) and one is kept active per bucket — the highest-scoring one stays, the rest are rotated out.

1.22.1 20a. Concepts

Term	Meaning
Round	One auto-tweaker invocation that produced an evaluable outcome (`BELOW_THRESHOLD`, `APPLIED`, `PROPOSED`, `DRY_RUN_WRITTEN`). `INELIGIBLE` and `ERROR` are NOT rounds.
Successful round	A round whose outcome is `BELOW_THRESHOLD`.
Streak	Consecutive successful rounds. Resets to 0 on any change-triggering outcome. Persisted in `state.json`.
Streak X	The configurable threshold for snapshot creation (`snapshot_streak_x`, default 3).
Active snapshot	The snapshot file representing the current settings during an ongoing streak. Status flips to `ACTIVE` in the manifest on creation, `ROTATED` when superseded by a higher-scoring snapshot in the same bucket.
Condition bucket	A categorical key for snapshot retention: `regime × volatility tier`. 12 buckets total.

1.22.2 20b. Streak tracking

`AutoTweakerCore` increments `state.CurrentBelowThresholdStreak` after each evaluable round:

- `BELOW_THRESHOLD`: streak += 1. When `streak == snapshot_streak_x` and no active snapshot exists, `SnapshotManager.Create()` fires. When the streak grows past X, `SnapshotManager.AccumulateConditions()` re-extracts the conditions vector across the full streak window and updates the manifest row in place.
- `APPLIED` / `PROPOSED` / `DRY_RUN_WRITTEN`: if an active snapshot exists, `SnapshotManager.Finalise()` populates `FinalisedIso` (the **last successful round's timestamp**, not this interrupting round's timestamp) and runs the bucket-rotation check. Streak resets to 0.
- `INELIGIBLE` / `ERROR`: no-op. Streak unchanged. Engine restart preserves the streak via `state.json`.

1.22.3 20c. Snapshot creation trigger

When the streak first hits `snapshot_streak_x`:

1. Copy `settings.json` verbatim to `settings_snapshots/settings_snapshot_<yyyyMMddHHmmss>.json`. No wrapper, no metadata — the file is a directly-applicable `settings.json`.
2. Compute the initial conditions vector via `ConditionsExtractor.Extract()` over the streak's CSV row range.
3. Append a manifest row with `Status = ACTIVE` and the full conditions vector. `FinalisedIso` stays empty until the streak is interrupted.

1.22.4 20d. Conditions vector

Stored in the manifest CSV (`settings_snapshots/manifest.csv`):

Field	Source
`RegimeMix`	Distribution of `Regime` column across the streak (pipe-format e.g. `UP:25|DN:10|RB:60|TR:5`).
`AtrScaleAvg` / `AtrScaleMin` / `AtrScaleMax`	Aggregates of the `ATRMultiplier` column (col 65).
`FundingMin` / `FundingMax`	Range of the `FundingRate` column (col 38).
`NetPriceMovePct`	`(lastPrice - firstPrice) / firstPrice × 100` over the streak window.
`VolumeRatioAvg`	Average of the `VolumeRatio` column (col 19).
`VerdictTierMix`	Distribution of `Verdict` column across the seven tier codes (SL/L/WL/NT/WS/S/SS).
`VWAPDevAvg` / `VWAPDevMin` / `VWAPDevMax`	Aggregates of `|VWAPDevPct|` (col 21).

Field	Source
`SpreadRegimeMix`	`SpreadBps` (col 69) bucketed by `cfg.Indicators.Spread.{Tight,Wide}ThresholdBps` into T / N / W.
`OFIImbalanceMix`	`OFISignal` (col 47) counted into BD / SD / BAL.
`ConditionBucket`	`{Regime}_{VolatilityTier}` where vol tier is `LOW` (<0.85), `NORMAL` (0.85..1.15), `HIGH` (≥1.15) of `AtrScaleAvg`.
`AvgFailureRatePct`	Average of round-level `AggregateFailureRatePct` across the streak.

The conditions are re-extracted on each accumulation and on finalisation — cleaner than incremental updates, cost is sub-millisecond for typical 360-row streaks.

1.22.5 20e. Composite score and bucket rotation

The composite score is used to compare snapshots within a bucket:

```
StreakLengthClamped = min(StreakLength, StreakLengthClamp)
Score = (100 - AvgFailureRatePct) + StreakLengthClamped * StreakWeight
```

Defaults: `StreakWeight = 1.5`, `StreakLengthClamp = 20`. Failure rate dominates (1 point per percentage point); streak adds a secondary nudge (1.5 per round, capped at 20).

Worked examples:

Snapshot	Fail %	Streak	Score	Notes
A	25%	5	82.5	Moderate both
B	35%	10	80.0	Higher fail, long streak
C	20%	8	92.0	Low fail, decent streak
D	15%	3	89.5	Lowest fail, short streak
E	40%	30 (→ 20)	90.0	Higher fail, very long streak

This exact formula must remain identical in `CompositeScorer.vb`, the `PromptBuilder` system message, and this section. Any change here requires the same edit in those two locations.

Rotation rule (run on each finalisation):

- If no existing `ACTIVE` snapshot in the same bucket, the new snapshot stays `ACTIVE`.
- Else, compute both scores fresh and compare:

- New > existing → existing is marked `ROTATED` (reason = `"superseded by <new.Filename>"`), its `.json` file is deleted (manifest row retained as historical record). The new snapshot stays `ACTIVE`.
- Otherwise → the new snapshot is immediately marked `ROTATED` (reason = `"score X <= existing Y"`), its `.json` file is deleted.

1.22.6 20f. Manifest schema

Single CSV at `settings_snapshots/manifest.csv` (gitignored). One row per snapshot. Columns:

```
Filename, CreatedIso, FinalisedIso, StreakLength, AvgFailureRatePct,
RegimeMix, AtrScaleAvg, AtrScaleMin, AtrScaleMax, FundingMin, FundingMax,
NetPriceMovePct, VolumeRatioAvg, VerdictTierMix,
VWAPDevAvg, VWAPDevMin, VWAPDevMax, SpreadRegimeMix, OFIImbalanceMix,
ConditionBucket, Status, RotationReason
```

`Status` is `ACTIVE` or `ROTATED`. `RotationReason` is empty unless rotated. Fields with embedded commas are RFC-4180 quoted.

1.22.7 20g. Revert mechanism

When a change-triggering round fires, the prompt to Claude is extended with the ACTIVE manifest rows and the current conditions vector. Claude may respond with either:

- `action="tweak"` — a normal diff (subject to the `max_keys_per_proposal` cap).
- `action="revert"` with a `revert_target` filename — a wholesale replacement from a past snapshot.

For a revert:

1. `SettingsDiffApplier.ApplyRevert()` is invoked. It validates that `revert_target` matches an `ACTIVE` manifest row and the snapshot file exists on disk.
2. The snapshot content is run through the **same rejected-pattern / disabled-gate validation** as a normal diff (banned fragments like `_fixed_pct_`, disabling `mtf_gate.enabled`, etc.) — snapshot integrity is not absolute trust.
3. The diff-scope cap is **NOT** applied to reverts — by definition a revert is many keys at once. The snapshot's provenance (a proven-successful streak) is the validation gate.
4. `settings.json` is replaced with the snapshot content, `version` bumped, `modified_by = "auto-tweaker-revert"`, and a `change_log` entry is appended citing the snapshot filename + reasoning excerpt.

Reverts honour the `auto_commit_enabled` toggle the same way tweaks do — when off, the revert

is written to `tools/AutoTweaker/proposed_diffs/<timestamp>_revert.json` for manual review.

1.22.8 20h. Round Statistics display

The Tweak Settings dialog exposes a `Show Round Stats` button that opens `RoundStatsForm` (non-modal). The form renders the last 5 rounds in `state.RoundHistory` with:

- Round timestamp + outcome
- Aggregate failure rate for the window
- For `APPLIED` / `PROPOSED` / `DRY_RUN_WRITTEN` rounds: the diff summary and a reasoning excerpt
- For each round, per-tier accuracy using the v2 barrier-hit semantic from `FailureRateMatrix.WalkBars`. This panel covers **all directional verdicts** (STRONG_LONG / LONG / WEAK_LONG / STRONG_SHORT / SHORT / WEAK_SHORT) — not just the tier-eligible subset used by the failure-rate matrix. `NO_TRADE` rows are reported as an informational row-count only.

OHLC bars for the row windows are fetched via `DeribitOhlcFetcher` once per `Refresh`. The build is async; expected cost is well under a second for 5 rounds × 120 rows.

The form also subscribes to the `RoundStatsForm.Refresh` button — re-run any time to pick up newer rounds.

1.22.9 20i. State persistence

`state.json` (gitignored, in `tools/AutoTweaker/`) carries the snapshot-related state across runs:

- `current_below_threshold_streak` — running streak counter
- `active_snapshot_filename` / `active_snapshot_created_iso` — pointer to the current ACTIVE snapshot
- `last_successful_round_iso` — timestamp of the most recent `BELOW_THRESHOLD` round, used to populate `FinalisedIso`
- `round_history` — last 50 `RoundSummary` entries (older entries dropped on save)

1.23 21. Live Performance Display

A compact strip of six labels positioned on the auto-run row (to the right of the Start/Stop button) that shows how the engine's directional verdicts have been performing, updated after every analysis run. The strip is read-only and observational — it does not feed into scoring or auto-tweaker decisions.

1.23.1 21a. Concepts

Six time windows:

Label	Window definition
`Cur.Wk`	Monday 00:00 UTC+8 → now
`3d`	D-2 00:00 UTC+8 → now
`Cur.Day`	Today 00:00 UTC+8 → now
`Asia`	Most-recent Asia block — 08:00–15:00 UTC+8
`London`	Most-recent London block — 16:00–20:00 UTC+8
`NY`	Most-recent NY block — 21:00–07:00 UTC+8 (midnight straddle)

Success rate = `SUCCESS_count / (SUCCESS_count + FAILURE_count) × 100`, expressed as an integer percent (e.g. `57%`). PENDING rows and EXCLUDED rows do not enter the denominator.

Sample-size threshold. When fewer than `min_sample_for_render` evaluable predictions exist in a window, the label shows `--%` (uncoloured). Default threshold: 4. Configurable via `performance_display.min_sample_for_render`.

1.23.2 21b. Most-recent-block algorithm

Each session label covers the most recently active or completed block for that session, regardless of calendar date. There are three states:

State	Displayed window
`now` is **inside** the session hours	Session start today → now (partial, growing)
`now` is **after** today's session	Today's full session block (start → end)
`now` is **before** today's session	Yesterday's full session block

The NY session straddles midnight UTC+8 (21:00–07:00). Three sub-cases:

- **Hour 21:00–23:59 UTC+8** — inside NY head. Block = today 21:00 → now.
- **Hour 00:00–06:59 UTC+8** — inside NY tail. Block = yesterday 21:00 → now.
- **Hour 07:00–20:59 UTC+8** — between sessions. Block = yesterday 21:00 → today 07:00 (completed).

Worked examples (UTC+8):

- `02:30 UTC+8` — Asia: yesterday 08:00–15:00 (completed). London: yesterday 16:00–20:00 (completed). NY: yesterday 21:00 → 02:30 (running, partial).
- `10:00 UTC+8` — Asia: today 08:00 → 10:00 (running). London: yesterday 16:00–20:00. NY: yesterday 21:00–07:00 today (completed, 10h block).

- `17:30 UTC+8` — Asia: today 08:00–15:00 (completed). London: today 16:00 → 17:30 (running). NY: yesterday 21:00–07:00 (completed).
- `22:00 UTC+8` — Asia: today 08:00–15:00. London: today 16:00–20:00. NY: today 21:00 → 22:00 (running).

The tooltip on each label confirms the exact range and sample size.

1.23.3 21c. Success metric — v2 barrier-hit

Reuses `FailureRateMatrix.WalkBars` from the v2 failure-definition spec.

Eligible verdicts: `STRONG LONG`, `LONG`, `WEAK LONG`, `STRONG SHORT`, `SHORT`, `WEAK SHORT`. `NO TRADE` and `NO TRADE [WEAK X]` are not counted.

Excluded rows (omitted from numerator and denominator): - `ATR` $\leq$ `0` — degenerate barriers. Marked `EXCLUDED_ATR_INVALID`. - Non-directional verdicts. Marked `EXCLUDED_NO_PREDICTION`.

Favourable barrier: - LONG direction: `AdjustedLongTarget` if > 0, else `entry + 2.0 × ATR`. - SHORT direction: `AdjustedShortTarget` if > 0, else `entry - 2.0 × ATR`.

Adverse barrier: - LONG: `SwingStopLong` if > 0, else `entry - 1.2 × ATR`. - SHORT: `SwingStopShort` if > 0, else `entry + 1.2 × ATR`.

Eligible bars: T+3 min through T+15 min (13 bars), skipping T+1 and T+2. Bars are identified by `CloseTime` in the OHLC cache (`CloseTime = openTime + 1 min`).

Classification: - `SUCCESS` — favourable wick hit before adverse hit. - `ADVERSE_HIT` — adverse hit before favourable hit → FAILURE. - `AMBIGUOUS` — both barriers hit in the same bar → FAILURE (conservative-bias rule). - `WINDOW_EXPIRED` — neither barrier hit by T+15 → FAILURE.

1.23.4 21d. Cache architecture

Two sidecar files in the same directory as `analysis_log.csv` (`bin/Debug/net8.0-windows/` or `bin/Release/…`). Both are gitignored.

`analysis_eval_cache.csv` Schema comment: `# schema=v1 (live-performance-display)`

Columns: `Timestamp,Verdict,EntryPrice,FavBar,AdvBar,EvalOutcome`

- One row per analysis run.
- `EvalOutcome` values: `SUCCESS`, `ADVERSE_HIT`, `AMBIGUOUS`, `WINDOW_EXPIRED`, `EXCLUDED_NO_PREDICTION`, `EXCLUDED_ATR_INVALID`, `PENDING`.
- `PENDING` rows are resolved in-place when their 15-min window completes. The file is rewritten when any PENDING row resolves (read-all → modify → write-back).

ohlc_1m_cache.csv Schema comment: `# schema=v1 (1m ohlc cache)`

Columns: `CloseTime,Open,High,Low,Close,Volume` (Volume = 0 placeholder).

- Rolling 7-day window: cap = 10,080 bars ($7 \times 24 \times 60$).
- Trim fires when the in-memory count exceeds cap $\times$ 1.05 (~10,584), batching disk writes to every ~500 bars rather than every bar.

1.23.5 21e. Cold-start backfill flow

On engine startup (when `eager_backfill_on_startup = true`):

1. **OHLC cache check.** If `ohlc_1m_cache.csv` exists and its newest bar is within 7 days, load it and fetch only the gap (last bar → now) via `DeribitClient.GetCandlesAsync`. Otherwise fetch the full 7-day range (~10,080 bars). Typical cost: 1–3 seconds.
2. **Eval cache backfill.** Walk `analysis_log.csv`. For each row not already in the eval cache, compute barriers and evaluate or mark PENDING.
3. **Initial display.** Once backfill completes, all 6 labels populate from the in-memory eval cache.
4. **Status.** `lblLogInfo` shows "Loading performance history…" during backfill. Controls remain responsive — backfill runs on a background thread. New analysis runs wait for backfill to complete via an internal init gate (typically < 5 seconds).

On subsequent launches the cached files load in under 1 second and the gap fetch covers only the time since the last save.

1.23.6 21f. Per-analysis update flow

After each `RunAnalysisAsync` completes (after `RenderOutput` and `AnalysisOutputDump.Append`):

1. **Append new OHLC bars.** Walk `candles1m` from oldest to newest. Bars with `CloseTime > cache_max` are appended to disk and added to the in-memory dictionary. Typically 1 new bar per analysis run.
2. **Append current verdict as PENDING.** Compute `FavBar` and `AdvBar` from live `VerdictResult` and `IndicatorResults`. Write the row with `EvalOutcome = PENDING`.
3. **Resolve PENDING rows.** Find all eval-cache rows with `EvalOutcome = PENDING` where `Timestamp + 15 min` $\leq$ `nowUtc`. For each, call `WalkBars` against the in-memory OHLC cache. At most 1 row resolves per run (the row from ~15 min ago). Rewrite the eval cache file.
4. **Recompute 6 window aggregates.** Pure in-memory scan; no additional I/O. Typical cost: < 5 ms.

5. **Update labels.** `UpdatePerformanceLabels()` applies colours and tooltip text on the UI thread.

1.23.7 21g. Render rules and tooltips

Label text: - Sample size ≥ threshold: `"{prefix}: {rate}%"` — e.g. `Asia: 64%` - Sample size < threshold: `"{prefix}: --%"` — e.g. `Asia: --%`

Colour: - Rate > 50% → `C_GOOD` (green, `#50DC78`) - Rate ≤ 50% → `C_BAD` (red, `#FF5050`) - `--%` → `C_DIM` (dim grey, `#646464`)

Tooltip on hover: `"{n} predictions evaluated. {start} → {end} UTC+8."`

Example: `12 predictions evaluated. 2026-05-13 08:00 → 2026-05-13 10:45 UTC+8.`

1.23.8 21h. Settings reference

Block: `performance_display` in `settings.json`.

Key	Type	Default	Description
`enabled`	bool	`true`	Master switch. `false` = strip hidden, no cache I/O.
`min_sample_for_render`	int	`4`	Minimum evaluable rows before showing a numeric rate.
`eager_backfill_on_startup`	bool	`true`	Fetch 7-day OHLC gap and backfill eval cache on launch.
`session_block_semantic`	string	`"most_recent"`	Reserved. Currently always most-recent-block (§21b).

Added in `settings.json` v26 (`modified_by = "live-performance-display"`).